Stack and Queue

Why are we studying Data Structures ?

Dealing with data…

- How to store it ?
- How to process it ?
- How to use it ?
- How to gain “knowledge” from it ?
- How to keep it secret?

Dealing with data…

- How to store it ?
- How to process it ?
- How to use it ?
- How to gain “knowledge” from it ?
- How to keep it secret?

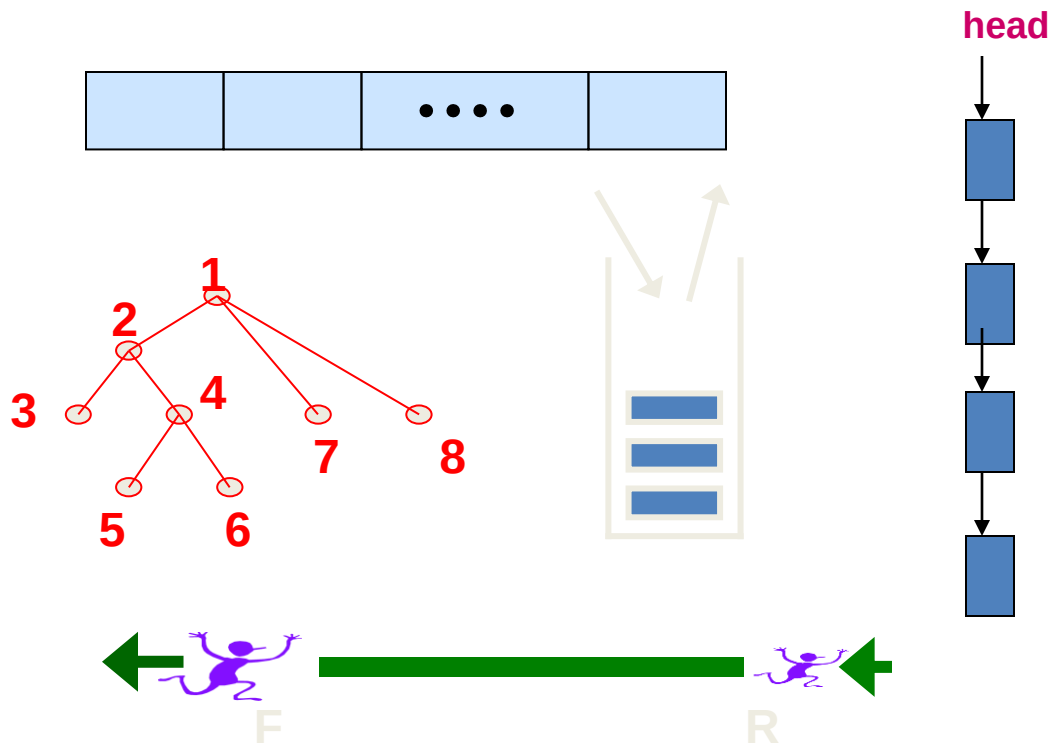
How should data be stored?

Depends on your requirement

"We back up our data on sticky notes because sticky notes never crash."

Elementary Data "Structures"

- **Arrays**
- **Lists**
- **Stacks**
- **Queues**
- **Trees**

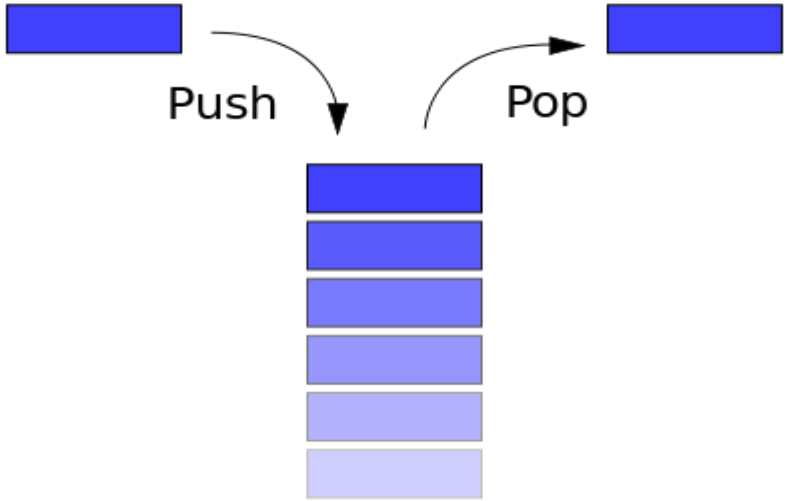


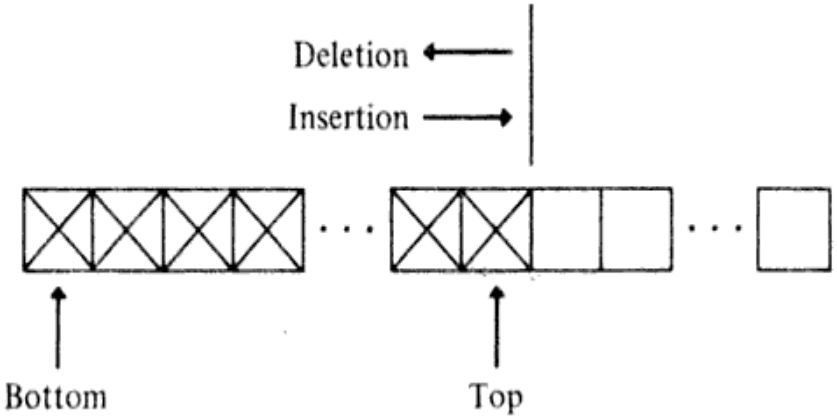
In some languages these are basic data types – in others they need to be implemented

Stacks

Introduction

- Stack is an important data structure which stores its elements in an ordered manner.
- A stack is a linear data structure which uses a principle, that the elements in a stack are added and removed only from one end, which is called the *top*.
- Hence, a stack is called a LIFO (Last-In, First-Out) data structure as the element that is inserted last is the first one to be taken out.





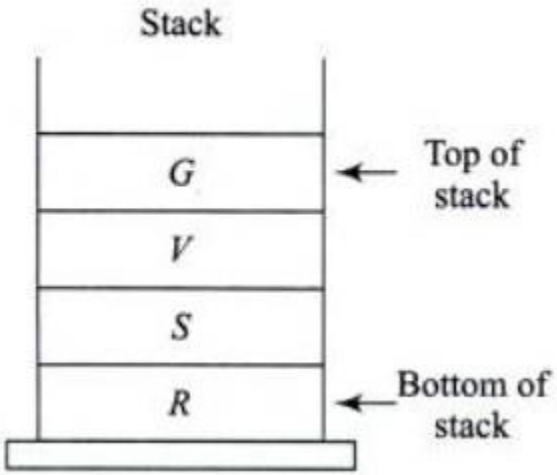
Introduction

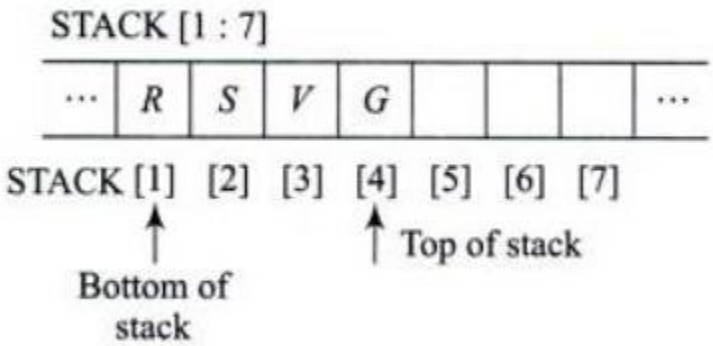
- Real life examples of stack:
  - Suppose we have created stack of the book
  - How books are arranged in the stack?
    - Books are kept one above the other
    - Books which are inserted first is taken out last. (brown)
    - Book which is inserted lastly is served first. (light green)
  - Suppose at your home you have multiple chairs, then you put them together to form a vertical pile. From that vertical pile, the chair placed last is always removed first.
  - Chair which was placed first will be removed last.

What is this good for ?

- Page-visited history in a Web browser
- Undo sequence in a text editor
- Saving local variables when one function calls another, and this one calls another

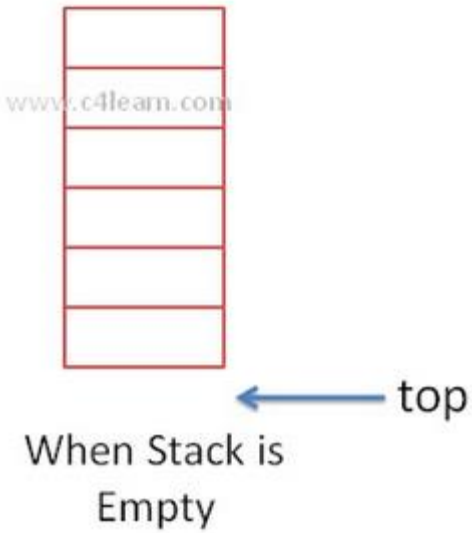
Array representation of stacks





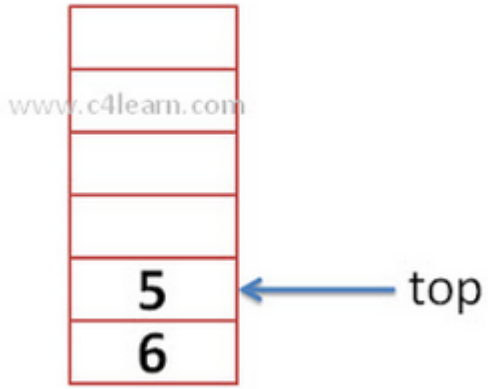
Visual Representation of Stack

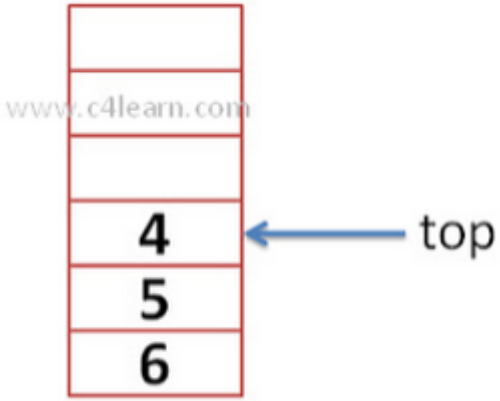
- View 1: When stack is empty
  - When stack is empty then it does not contain any element inside it. Whenever stack is empty, the position of topmost element is -1.



Visual Representation of Stack

- View 2: When stack is not empty
  - Whenever we add very first element then topmost position will be increment by 1. After adding first element, top = 0.





- View 3: After deletion of 1 element top will be decremented by 1.

Visual Representation of Stack

- Position of top and its value:

Position of Top	Status of Stack
-1	Stack is Empty
0	First Element is Just Added into Stack
N-1	Stack is said to Full
N	Stack is said to be Overflow

- Values of stack and top:

Operation	Explanation
top = -1	-1 indicated Empty Stack
top = top + 1	After push operation value of top is incremented by integer 1
top = top – 1	After pop operation value of top is decremented by 1

How should we represent it ?

- Write code in python ?
- Write code in C ?
- Write code in Java ?

Aren’t we essentially doing the same thing?

Abstract Data Type

A mathematical definition of objects, with operations defined on them

Examples

- Basic Types
  - integer, real (floating point), boolean (0,1), character
- Arrays
  - A[0..99] : integer array

	0	1	2	3	4	5	6	7					99
A	2	1	3	3	2	9	9	6	…				10

  - A[0..99] : array of images

ADT: Array

A mapping from an index set, such as `{0,1,2,…,n}`, into a cell type

Objects: set of cells

Operations:

- `create(A,n)`
- `put(A,v,i)` or `A[i] = v`
- `value(A,i)`

Abstract Data Types (ADTs)

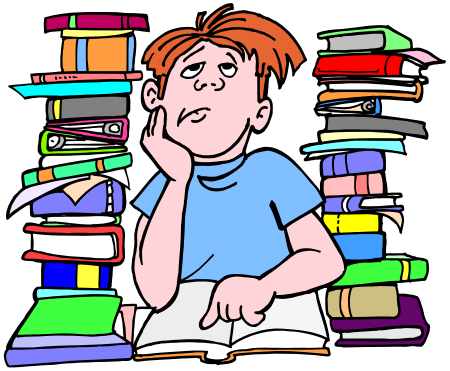
- An abstract data type (ADT) is an abstraction of a data structure

- An ADT specifies:
  - Data stored
  - Operations on the data
  - Error conditions associated with operations

ADT for stock trade

- The data stored are buy/sell orders
- The operations supported are
  - order buy (stock, shares)
  - order sell (stock, shares)
  - void cancel (order)
- Error conditions:
  - Buy/sell a nonexistent stock
  - Cancel a nonexistent order

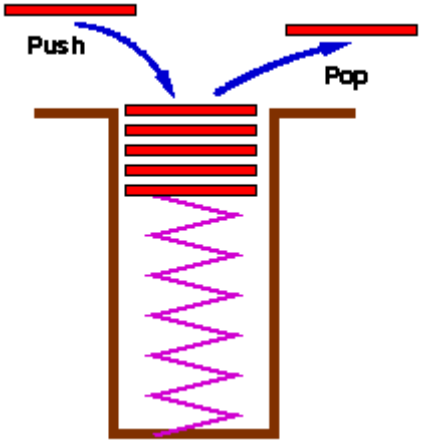
Stack ADT



Objects:

A finite sequence of nodes

Operations:



- Create
- Push: Insert element at top
- Top: Return top element
- Pop: Remove and return top element
- IsEmpty: test for emptyness

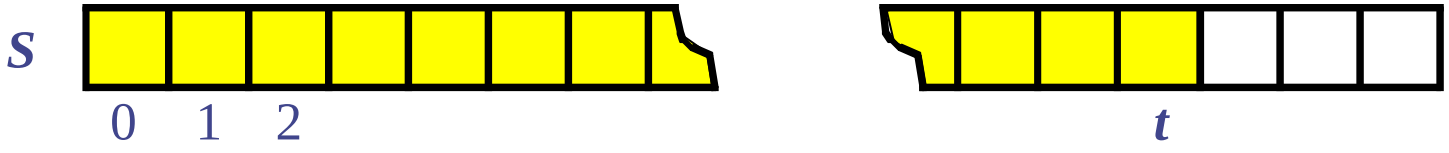
Exceptions

- Attempting the execution of an operation of ADT may sometimes cause an error condition, called an exception
- Exceptions are said to be “thrown” by an operation that cannot be executed
- In the Stack ADT, operations pop and top cannot be performed if the stack is empty
- Attempting the execution of pop or top on an empty stack throws an EmptyStackException

Array-based Stack

- A simple way of implementing the Stack ADT uses an array
- We add elements from left to right
- A variable keeps track of the index of the top element

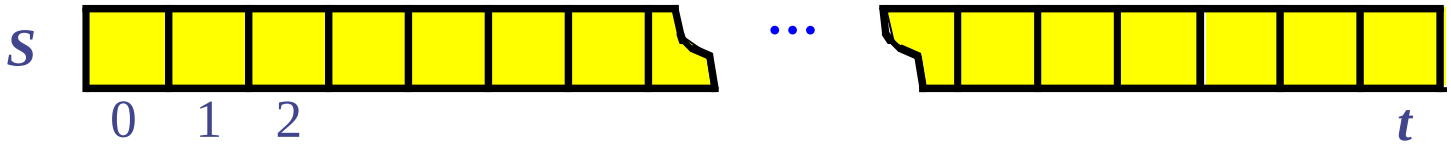
```
Algorithm size()
  return t + 1

Algorithm pop()
  if empty() then
    throw EmptyStackException
   else
    t = t - 1
    return S[t + 1]
```



Array-based Stack (cont.)

- The array storing the stack elements may become full
- A push operation will then throw a FullStackException
  - Limitation of the array-based implementation
  - Not intrinsic to the Stack ADT

```
Algorithm push(o)
  if t = S.length - 1 then
    throw FullStackException
   else
    t = t + 1
    S[t] = o
```



Performance and Limitations

- array-based implementation of stack ADT

- Performance
  - Let $\boldsymbol{n}$ be the number of elements in the stack
  - The space used is $\boldsymbol{O}(\boldsymbol{n})$
  - Each operation runs in time $\boldsymbol{O}(1)$
- Limitations
  - The maximum size of the stack must be defined *apriori* and cannot be changed
  - Trying to push a new element into a full stack causes an implementation-specific exception

Growable Array-based Stack

- In a push operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one
- How large should the new array be?
  - incremental strategy: increase the size by a constant c
  - doubling strategy: double the size

```
Algorithm push(o)
  if t = S.length - 1
then
    A = new array of
          size …
    for i = 0 to t do
       A[i] = S[i]
       S = A
  t = t + 1
  S[t] = o
```

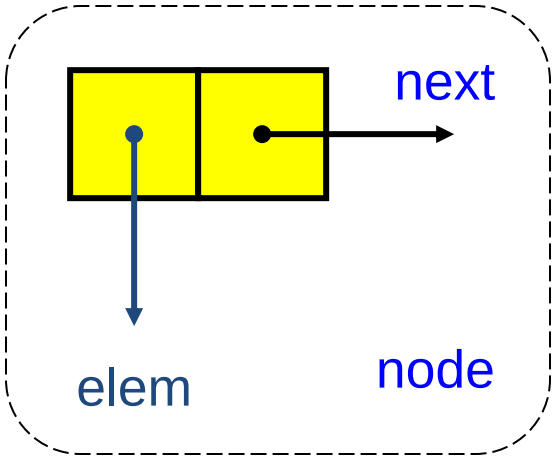
Stack Interface

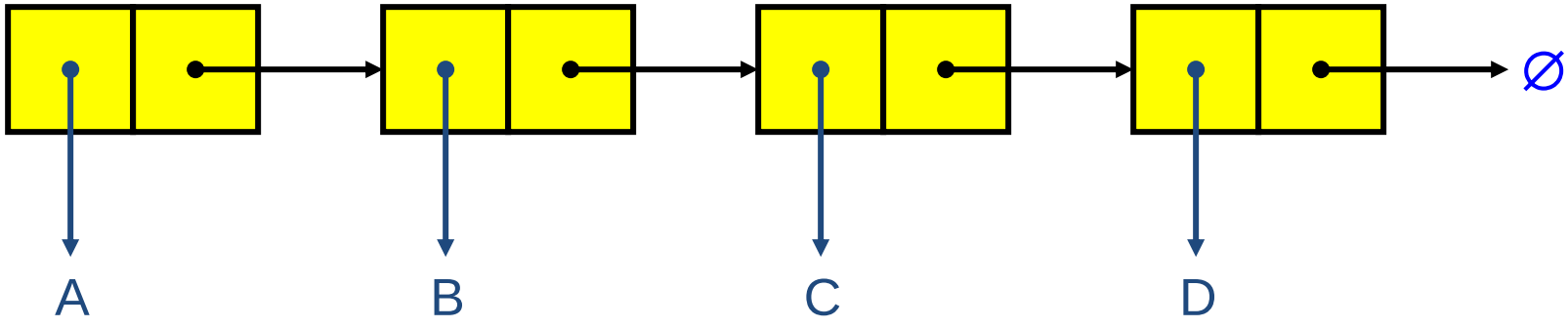
- Interface corresponding to our Stack ADT
- Requires the definition of class EmptyStackException

```
class Stack {
public:
    int size()
    bool isEmpty()
    Object& top()
            throw(EmptyStackException)
    void push(Object o)
    Object pop()
            throw(EmptyStackException);
};
```

Singly Linked List

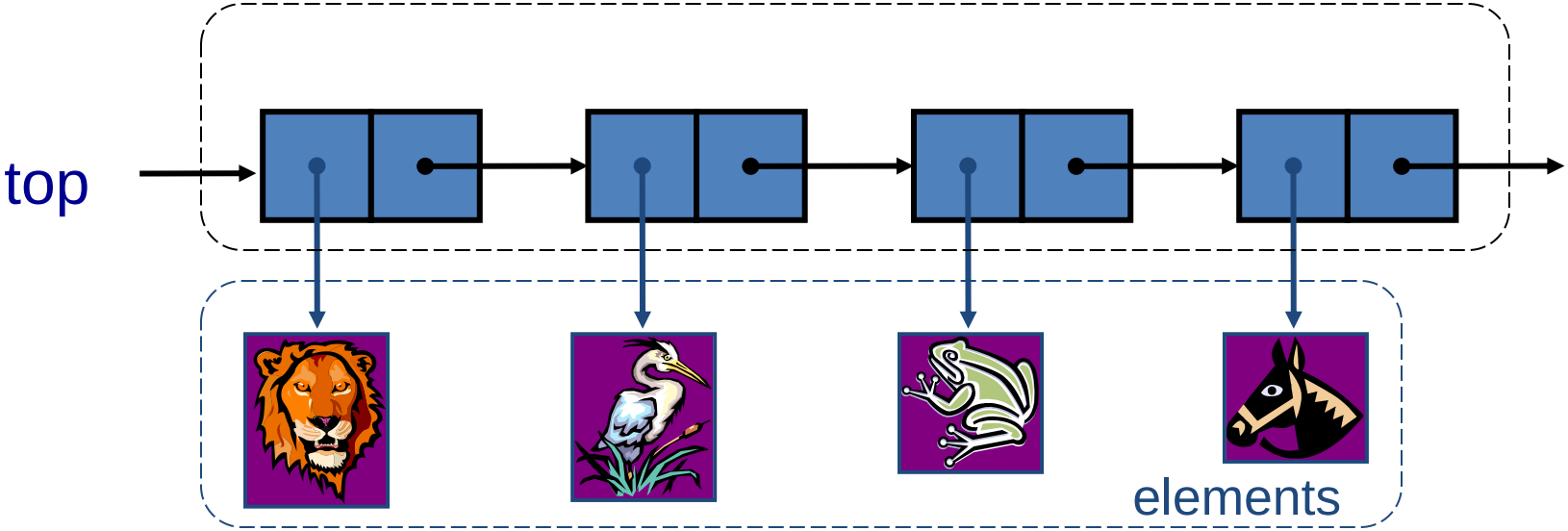
- A singly linked list is a concrete data structure consisting of a sequence of nodes
- Each node stores
  - element
  - link to the next node





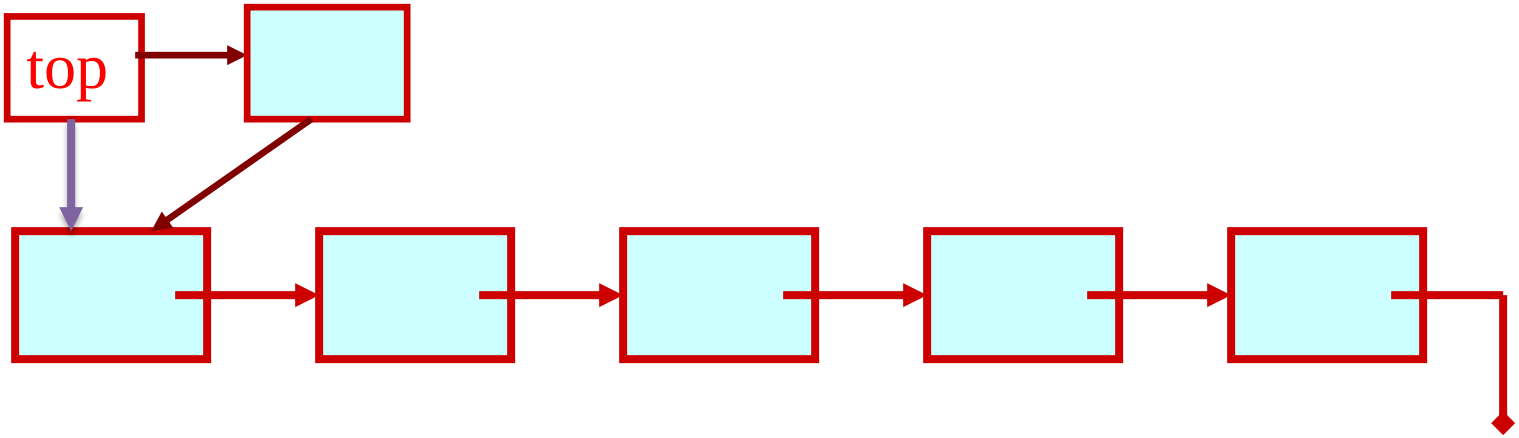
Stack with a Singly Linked List

- We can implement a stack with a singly linked list
- The top element is stored at the first node of the list
- The space used is $\boldsymbol{O}(\boldsymbol{n})$ and each operation of the Stack ADT takes $\boldsymbol{O}(1)$ time



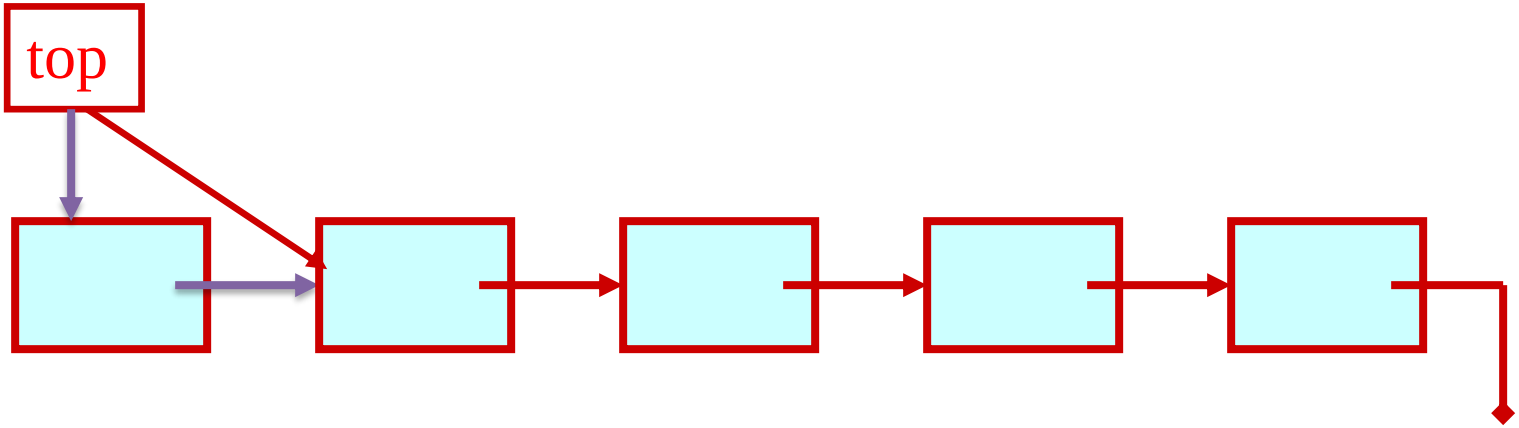
Push using Linked List

PUSH OPERATION



Pop using Linked List

POP OPERATION



Declaration

```
#define MAXSIZE 100

struct lifo
{
   int st[MAXSIZE];
   int  top;
};
typedef struct lifo
                 stack;
stack s;
```

ARRAY

```
struct lifo
{
   int value;
   struct lifo *next;
};
typedef struct lifo
                 stack;

stack *top;
```

LINKED LIST

Stack Creation

```
void create (stack *s)
{
   s->top = -1;

   /* s->top points to
      last element
      pushed in;
      initially -1 */
}
```

ARRAY

```
void create (stack **top)
{
   *top = NULL;

   /* top points to NULL,
      indicating empty
      stack              */
}
```

LINKED LIST

Pushing an element into stack

```
void push (stack *s, int element)
  {
   if (s->top == (MAXSIZE-1))
   {
     printf ("\n Stack overflow");
         exit(-1);
     }
     else
     {
         s->top++;
         s->st[s->top] = element;
     }
  }
```

ARRAY

```
void push (stack **top, int element)
{
    stack *new;

    new = (stack *)malloc (sizeof(stack));
    if (new == NULL)
    {
       printf ("\n Stack is full");
       exit(-1);
    }

    new->value = element;
    new->next = *top;
    *top = new;
}
```

LINKED LIST

Popping an element from stack

```
int pop (stack *s)
  {
     if (s->top == -1)
     {
        printf ("\n Stack underflow");
        exit(-1);
     }
     else
     {
        return (s->st[s->top--]);
     }
  }
```

ARRAY

```
int pop (stack **top)
{
   int t;
   stack *p;
   if (*top == NULL)
   {
      printf ("\n Stack is empty");
      exit(-1);
   }
   else
   {
      t = (*top)->value;
      p = *top;
      *top = (*top)->next;
      free (p);
      return t;
   }
}
```

LINKED LIST

Checking for stack empty

```
int isempty (stack *s)
{
   if (s->top == -1)
          return 1;
   else
          return (0);
}
```

ARRAY

```
int isempty (stack *top)
{
   if (top == NULL)
        return (1);
    else
        return (0);
}
```

LINKED LIST

Checking for Stack Full

```
int isfull (stack *s)
{
   if (s->top == (MAXSIZE-1))
   {
          return 1;
   else
          return (0);
   }
}
```

ARRAY

Example: A Stack using an Array

```
#include <stdio.h>
#define MAXSIZE 100

struct lifo
{
   int st[MAXSIZE];
   int  top;
};
typedef struct lifo stack;

main() {
  stack A, B;
  create(&A);
  create(&B);
  push(&A,10);
  push(&A,20);
  push(&A,30);
  push(&B,100);
  push(&B,5);

  printf("%d %d", pop(&A), pop(&B));

  push (&A, pop(&B));

  if (isempty(&B))
    printf ("\n B is empty");
  return;
}
```

Example: A Stack using Linked List

```
#include <stdio.h>
struct lifo
{
   int value;
   struct lifo *next;
};
typedef struct lifo stack;

main() {
  stack *A, *B;
  create(&A);
  create(&B);
  push(&A,10);
  push(&A,20);
  push(&A,30);
  push(&B,100);
  push(&B,5);

  printf("%d %d", pop(&A), pop(&B));

  push (&A, pop(&B));

  if (isempty(B))
    printf ("\n B is empty");
  return;
}
```

Infix and Postfix Notations

- Infix: operators placed between operands:

  A+B*C

- Postfix: operands appear before their operators:-

  ABC*+

- There are no precedence rules to learn in postfix notation, and parentheses are never needed

Infix to Postfix

Infix	Postfix
A + B	A B +
A + B * C	A B C * +
(A + B) * C	A B + C *
A + B * C + D	A B C * + D +
(A + B) * (C + D)	A B + C D + *
A * B + C * D	A B * C D * +

A + B * C → (A + (B * C)) → (A + (B C *)) → A B C * +

A + B * C + D → ((A + (B * C)) + D) → ((A + (B C*))+ D) → ((A B C *+) + D) → A B C * + D +

Infix to postfix conversion

- Use a stack for processing operators (push and pop operations).
- Scan the sequence of operators and operands from left to right and perform one of the following:
  - output the operand,
  - push an operator of higher precedence,
  - pop an operator and output, till the stack top contains operator of a lower precedence and push the present operator.

Operator precedence and associativity

Precedence	Operator	Associativity
1	**(), [], ., ->, ++ , --**	L->R
2	**++ / --, + / –, ! , ~, (type), *, &, sizeof**	R->L
3	***, /, %**	L->R
4	**+/-**	L->R
5	**<< , >>**	L->R
6	**< , <=, > , >=**	L->R
7	**== , !=**	L->R
8	**&**	L->R
9	**^**	L->R
10	**|**	L->R
11	**&&**	L->R
12	**||**	L->R
13	**?:**	R->L
14	**=, += , -=, *= , /=, %= , &=, ^= , |=, <<=, >>=**	R->L
15	**,**	L->R

The algorithm steps

1. Print operands as they arrive.
2. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
3. If the incoming symbol is a left parenthesis, push it on the stack.
4. If the incoming symbol is a right parenthesis, pop the stack and print the operators until you see a left parenthesis. Discard the pair of parentheses.
5. If the incoming symbol has higher precedence than the top of the stack, push it on the stack.
6. If the incoming symbol has equal precedence with the top of the stack, use association. If the association is left to right, pop and print the top of the stack and then push the incoming operator. If the association is right to left, push the incoming operator.
7. If the incoming symbol has lower precedence than the symbol on the top of the stack, pop the stack and print the top operator. Then test the incoming operator against the new top of stack.
8. At the end of the expression, pop and print all operators on the stack. (No parentheses should remain.)

Infix to Postfix Conversion

Requires operator precedence information

Operands:

Add to postfix expression.

Close parenthesis:

pop stack symbols until an open parenthesis appears.

Operators:

Pop all stack symbols until a symbol of lower precedence appears. Then push the operator.

End of input:

Pop all remaining stack symbols and add to the expression.

Infix to Postfix Rules

Expression:

A * (B + C * D) + E

becomes

A B C D * + * E +

Postfix notation is also called as Reverse Polish Notation (RPN)

	Current symbol	Operator Stack	Postfix string
1	A		A
2	*	*	A
3	(	* (	A
4	B	* (	A B
5	+	* (+	A B
6	C	* (+	A B C
7	*	* (+ *	A B C
8	D	* (+ *	A B C D
9	)	*	A B C D * +
10	+	+	A B C D * + *
11	E	+	A B C D * + * E
12			A B C D * + * E +

Postfix Evaluator

- Step 1: If char read from postfix expression is an operand, push operand to stack.
- Step 2: If char read from postfix expression is an operator, pop the first 2 operand in stack and implement the expression using the following operations:
  - **pop(opr1) then pop(opr2)**
  - **result = opr2 operator opr1**
- Step 3: Push the result of the evaluation to stack.
- Step 4: Repeat steps 1 to steps 3 until end of postfix expression
- Finally, At the end of the operation, only one value left in the stack. The value is the result of postfix evaluation.

Postfix Evaluator

postfix	Ch	Opr	Opn1	Opn2	result	stack
2 7 * 18 – 6 +						
7 * 18 – 6 +	2					2
* 18 – 6 +	7					2 7
18 – 6 +	*	*	7	2	14	14
– 6 +	18					14 18
6 +	–	–	18	14	-4	-4
+	6					-4 6
	+	+	6	-4	2	2

Prefix Evaluator

- Step 1: Reverse the prefix expression.
- Step 2: Read this reversed prefix expression from left to right one character at a time.
- Step 3: If char read from reversed prefix expression is an operand, push operand to stack.
- Step 4: If char read from reversed prefix expression is an operator, pop the first 2 operand in stack and implement the expression using the following operations:
  - **pop(opr1) then pop(opr2)**
  - **result = opr1 operator opr2**
- Step 5: Push the result of the evaluation to stack.
- Step 6: Repeat steps 3 to steps 5 until end of reversed prefix expression
- Finally, At the end of the operation, only one value left in the stack. The value is the result of prefix evaluation.

Prefix Evaluator

Example: -*+4325

Symbol	opnd1	opnd2	value	opndstack
5				5
2				5, 2
3				5, 2, 3
4				5, 2, 3, 4
+	4	3	7	5, 2
				5, 2, 7
*	7	2	14	5
				5, 14
-	14	5	9	
				9

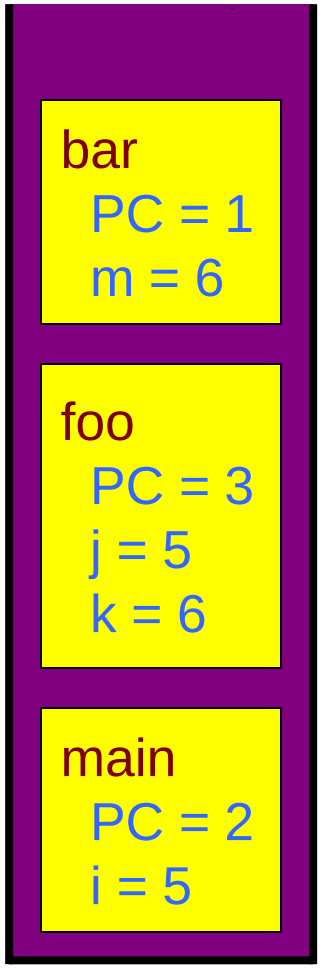
result

Exercise: Stacks

- Describe the output of the following series of stack operations
  - Push(8)
  - Push(3)
  - Pop()
  - Push(2)
  - Push(5)
  - Pop()
  - Pop()
  - Push(9)
  - Push(1)

Run-time Stack

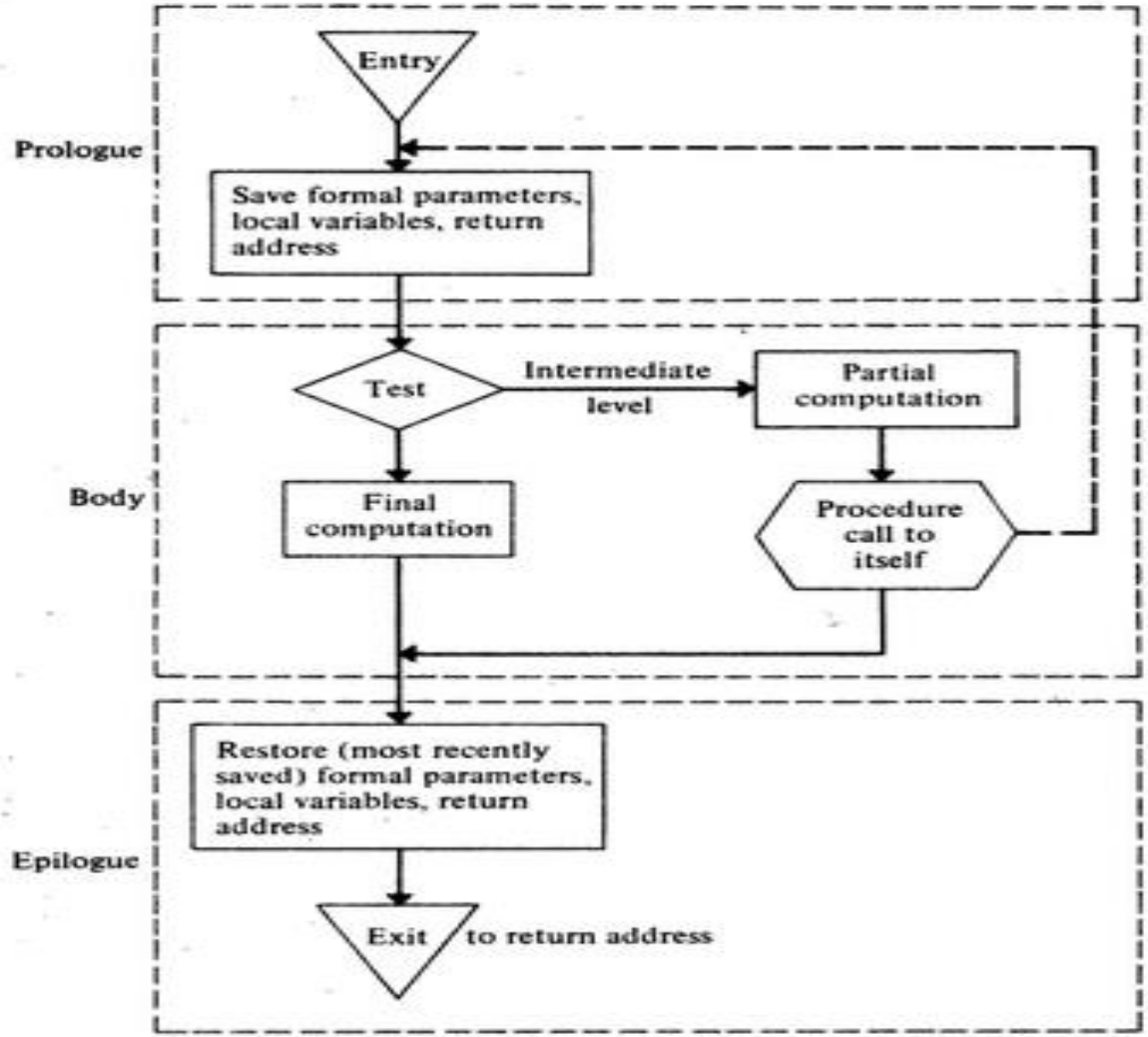
- The run-time system keeps track of the chain of active functions with a stack
- When a function is called, the run-time system pushes a frame on the stack containing
  - Local variables, including parameters
  - Program counter, keeping track of the statement being executed
- When a function returns, its frame is popped from the stack, and control is passed to the method on top of the stack



Recursion

- "Recursion" is a technique of solving any problem by calling the same function itself repeatedly until some breaking (base) condition is met where recursion stops and it starts calculating the solution from there on. For e.g. calculating the factorial of a given number
- Thus, in recursion the last function called needs to be completed first.
- Now Stack is a LIFO data structure i.e. (Last In First Out) and hence it is used to implement recursion.
- In each recursive call, there is a need to save the
  - Current values of parameters,
  - Local variables and
  - The return address (the address where the control has to return from the call).
- Also, as a function calls another function, first its arguments, then the return address, and finally, space for local variables is pushed onto the stack.

Recursion


Entry
Prologue
Save formal parameters, local variables, return address
Test
Intermediate level
Partial computation
Body
Final computation
Procedure call to itself
Restore (most recently saved) formal parameters, local variables, return address
Epilogue
Exit
to return address

Recursion

Factorial_I
Input: An integer number N.
Output: The factoral value of N, that is $N!$.
Remark: Code using the iterative definition of factorial.

Steps:

```
1. fact =1
2. For (i = 1 to N) do
3.     fact = i * fact
4. EndFor
5. Return(fact)                                   // Return the result
6. Stop
```

Here, Step 2 defines the iterative definition for the calculation of a factorial. Now, let us see the recursive definition of the same.

Factorial_R //Code using the recursive definition of factorial
Input: An integer number N.
Output: The factoral value of N, that is $N!$.
Remark: Code using the recursive definition of factorial.

Steps:

```
1. If (N = 0) then                                // Termination condition of repetition
2.     fact = 1
3. Else
4.     fact = N * Factorial_R(N − 1)
5. EndIf
6. Return(fact)                                   // Return the result
7. Stop
```

Here, Step 4 recursively defines the factorial of an integer N. This is actually a direct translation of $n! = n * (n - 1)!$ in the form of $Factorial_R\ (n) = n * Factorial_R\ (n - 1)$. This definition implies that $n!$ will be calculated if $(n - 1)!$ is known, which in turn if $(n - 2)!$ is known and so on until $n = 0$ when it returns 1 (Step 1).

Recursion

Steps:

```
 1.  5! = 5*4!
 2.          4! = 4*3!
 3.                  3! = 3*2!
 4.                          2! = 2*1!
 5.                                  1! = 1*0!
 6.                                          0! = 1
 7.                                  1! = 1
 8.                          2! = 2
 9.                  3! = 6
10.          4! = 24
11.  5! = 120
```

- Here, it is required to push the intermediate calculations till the terminal condition is reached. In the above calculation for 5!, Steps 1 to 6 are the push operations. The subsequent pop operations will evaluate the value of intermediate calculations till the stack is exhausted.
- Stack for parameter(s): To store the parameter with which the recursion is defined.
- Stack for local variable(s): To hold the local variable that are used within the definition.
- Stack to store the return address.

Tower of Hanoi Problem

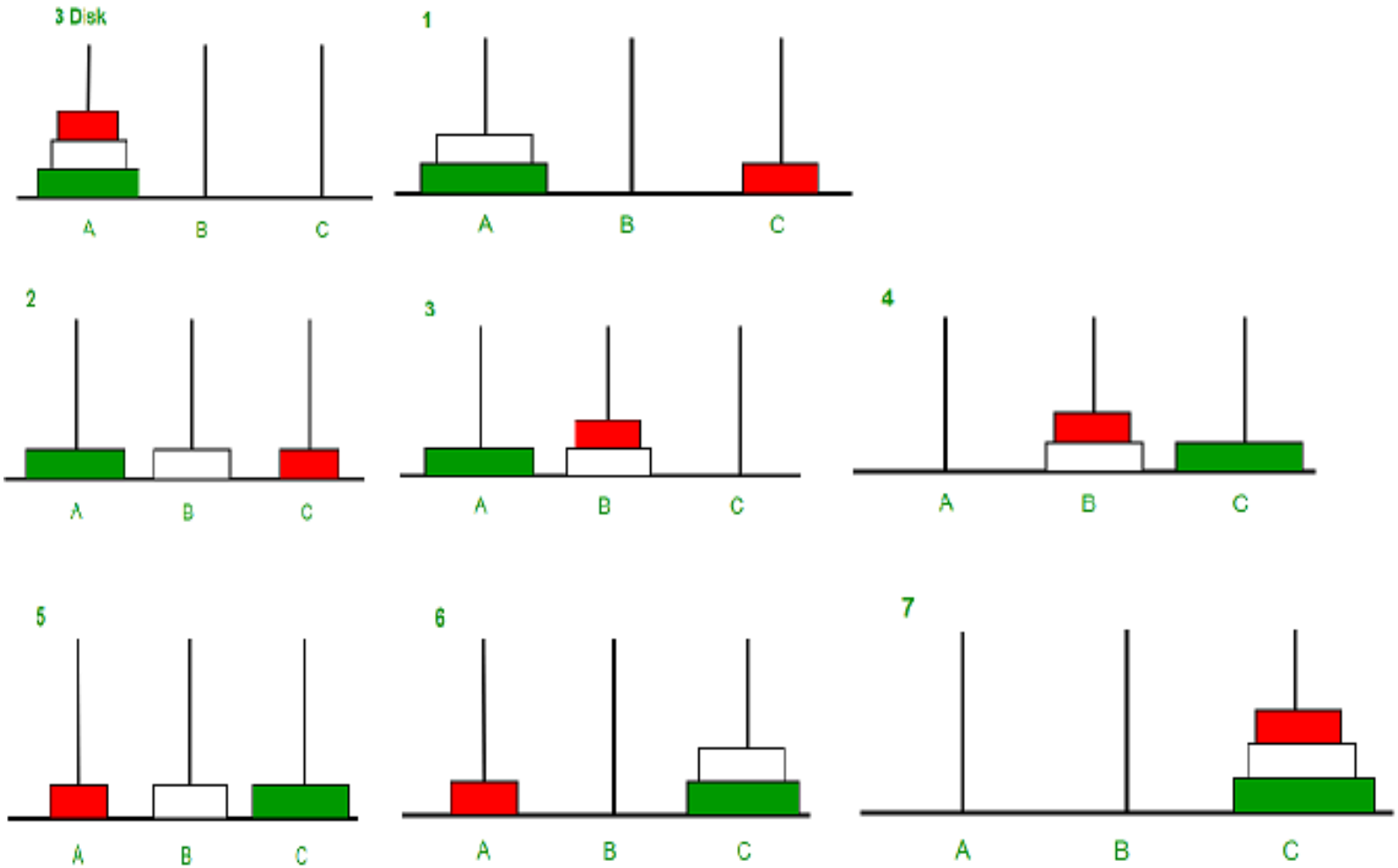
- The Tower of Hanoi puzzle is solved by moving all the disks to another tower by not violating the sequence of the arrangements.

The rules to be followed by the Tower of Hanoi are -

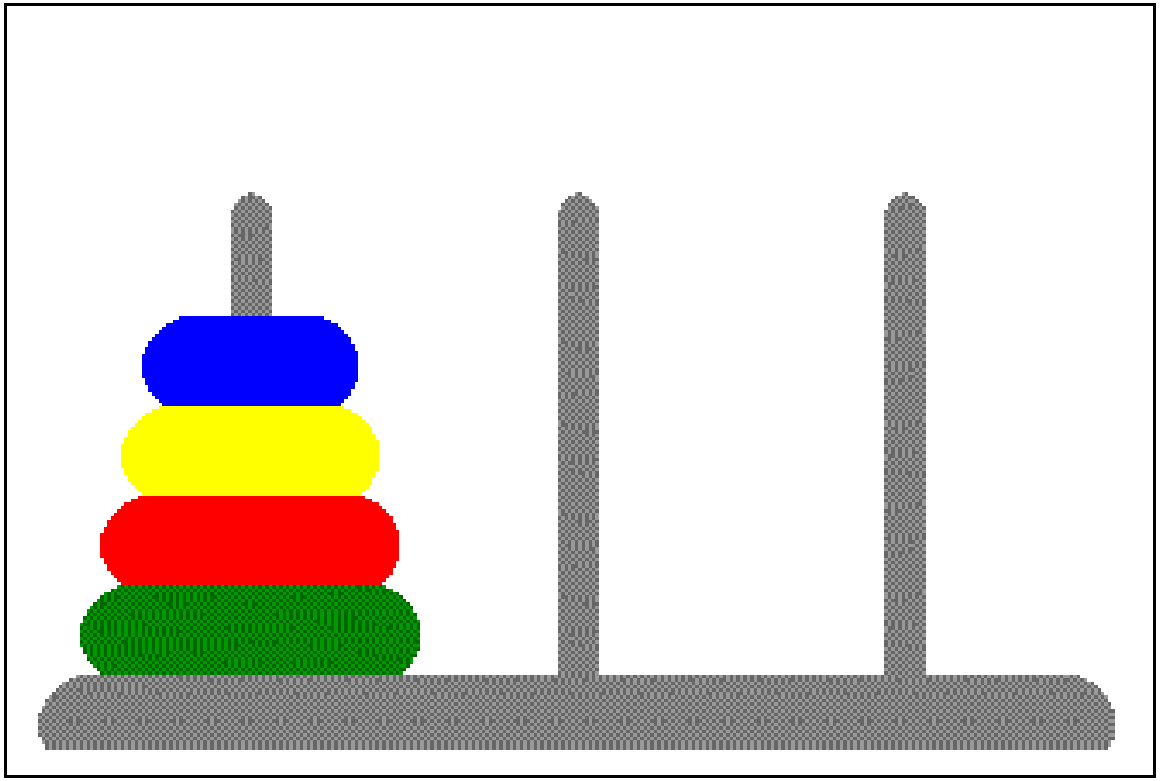
- Only one disk can be moved among the towers at any given time.
- Only the "top" disk can be removed.
- No large disk can sit over a small disk.

- **Algorithm**
- Step 1 – Move n-1 disks from source to aux (recursively shift)
- Step 2 – Move n^{th} disk from source to dest
- Step 3 – Move n-1 disks from aux to dest (recursively shift)

Example



Example



Reversing a Word

- We can use a stack to reverse the letters in a word.
- How?
- Read each letter in the word and push it onto the stack
- When you reach the end of the word, pop the letters off the stack and print them out.

Parentheses Matching

- Each “(”, “{”, or “[” must be paired with a matching “)”, “}”, or “[”
  - correct: ( )(( )){([( )])}
  - correct: ((( )(( )){([( )])}))
  - incorrect:)(( )){([( )])}
  - incorrect: ({[ ])}
  - incorrect: (

Parentheses Matching Algorithm

Algorithm ParenMatch(X,n):
Input: An array X of n tokens, each of which is either a grouping symbol, a variable, an arithmetic operator, or a number
Output: **true** if and only if all the grouping symbols in X match

```
Let S be an empty stack
for i=0 to n-1 do
    if X[i] is an opening grouping symbol then
        S.push(X[i])
    else if X[i] is a closing grouping symbol then
        if S.isEmpty() then
            return false {nothing to match with}
        if S.pop() does not match the type of X[i] then
            return false {wrong type}
if S.isEmpty() then
    return true {every symbol matched}
else
    return false {some symbols were never matched}
```

Exercise

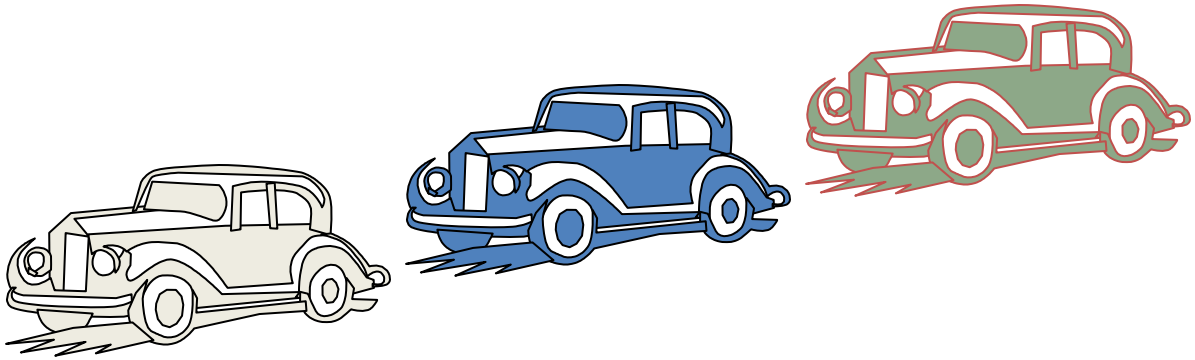
- Describe how to implement a stack using a singly-linked list
  - Stack operations: push(x), pop( ), size(), isEmpty()
  - For each operation, give the running time

Stack Summary

- Stack Operation Complexity for Different

	Array Fixed-Size	Array Expandable (doubling strategy)	List Singly-Linked
Pop()	O(1)	O(1)	O(1)
Push(o)	O(1)	O(n) Worst Case O(1) Best Case O(1) Average Case	O(1)
Top()	O(1)	O(1)	O(1)
Size(), isEmpty()	O(1)	O(1)	O(1)

Queues



Outline and Reading

- The Queue ADT
- Implementation with an array
  - Growable array-based queue
- List-based queue

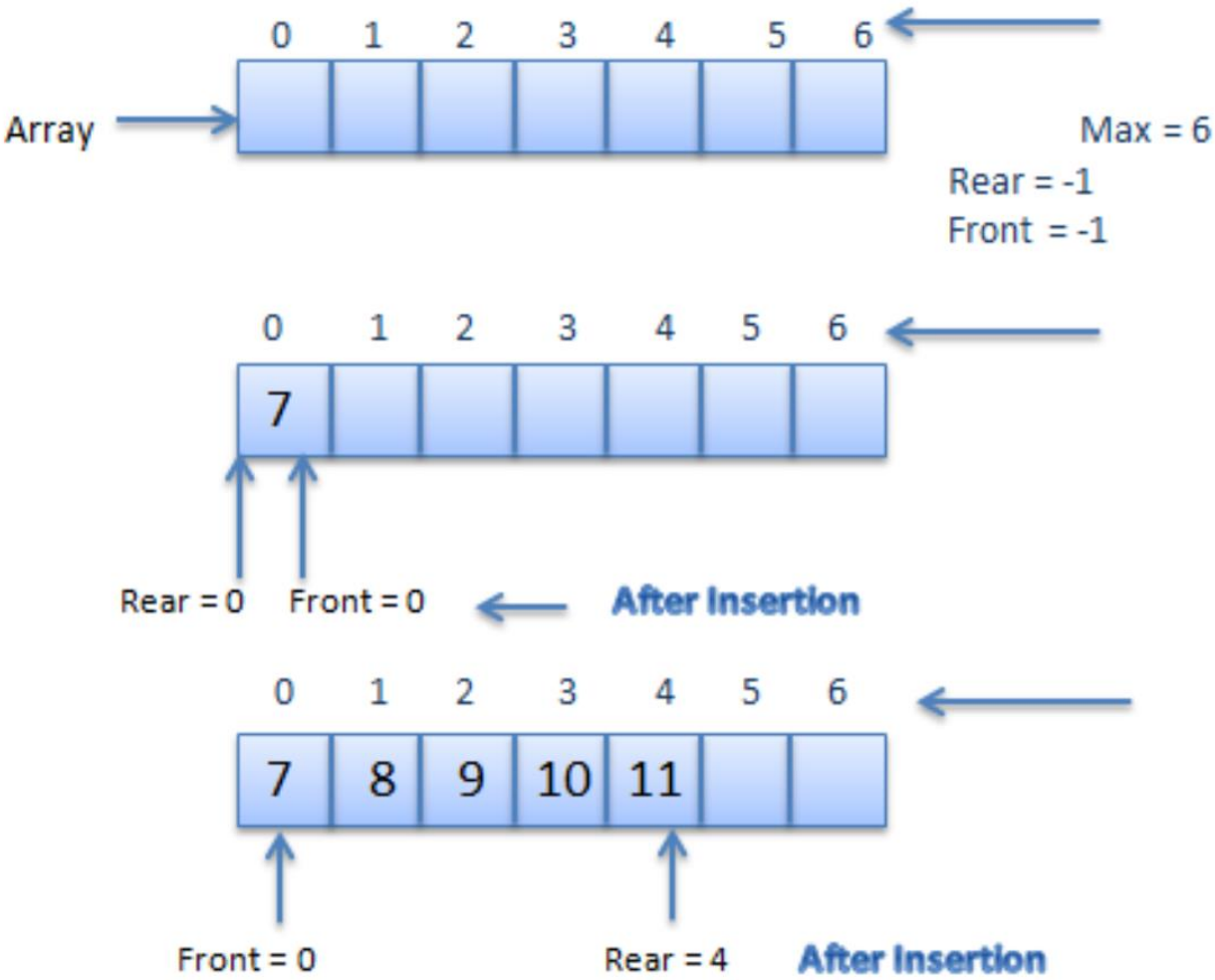
Basic Idea

- Queue is an abstract data structure, somewhat similar to Stacks. Unlike stacks, a queue is open at both its ends. One end is always used to insert data (enqueue) and the other is used to remove data (dequeue).

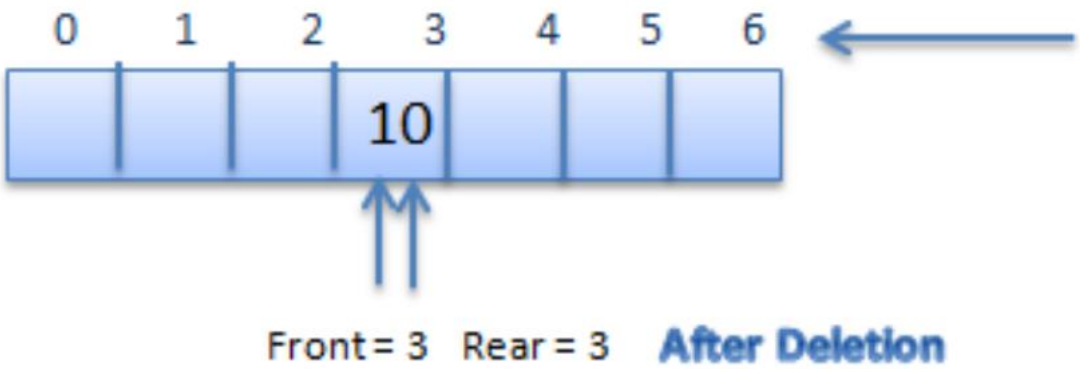
Queue Representation

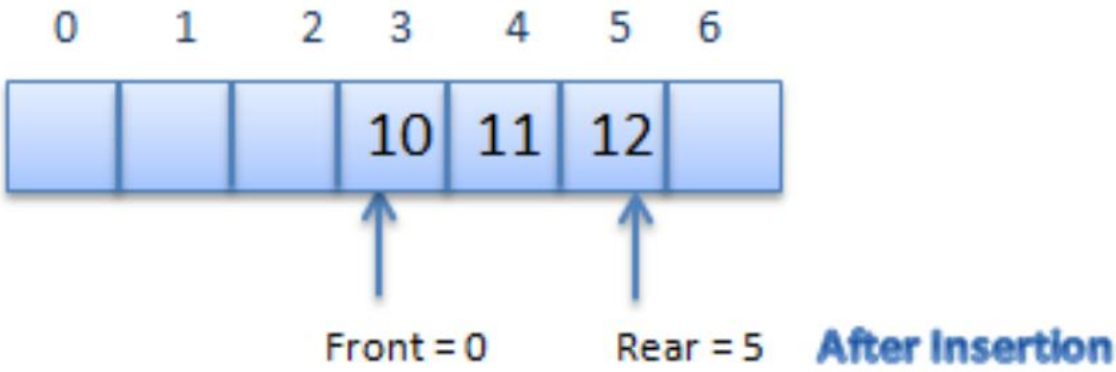
- As in stacks, a queue can also be implemented using Arrays, Linked-lists, Pointers and Structures.

Queue Operations

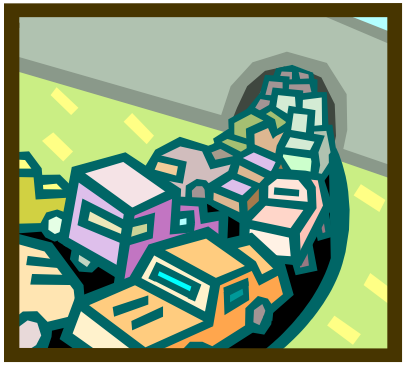


Queue Operations





Applications of Queues



- Direct applications
  - Waiting lines
  - Access to shared resources (e.g., printer)
  - Process Scheduling
  - Packet forwarding in router
  - Communication buffer
- Indirect applications
  - Auxiliary data structure for algorithms
  - Component of other data structures

The Queue ADT

- The Queue ADT stores arbitrary objects
- Insertions and deletions follow the first-in first-out (FIFO) scheme
- Insertions are at the rear of the queue and removals are at the front of the queue
- Main queue operations:
  - enqueue(object o): inserts element o at the end of the queue
  - dequeue(): removes and returns the element at the front of the queue
- Auxiliary queue operations:
  - front(): returns the element at the front without removing it
  - size(): returns the number of elements stored
  - isEmpty(): returns a Boolean value indicating whether no elements are stored
- Exceptions
  - Attempting the execution of dequeue or front on an empty queue throws an EmptyQueueException

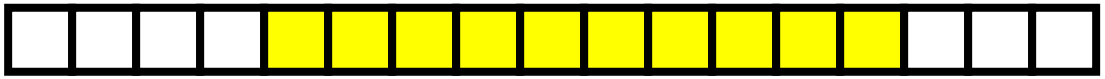
Exercise: Queues

- Describe the output of the following series of queue operations
  - enqueue(8)
  - enqueue(3)
  - dequeue()
  - enqueue(2)
  - enqueue(5)
  - dequeue()
  - dequeue()
  - enqueue(9)
  - enqueue(1)

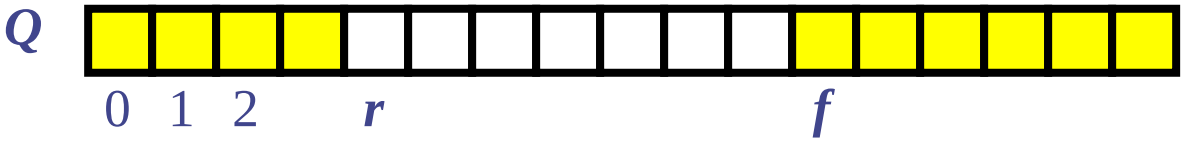
Array-based Queue

- Use an array of size N in a circular fashion
- Two variables keep track of the front and rear
  - f index of the front element
  - r index immediately past the rear element
- Array location r is kept empty

normal configuration



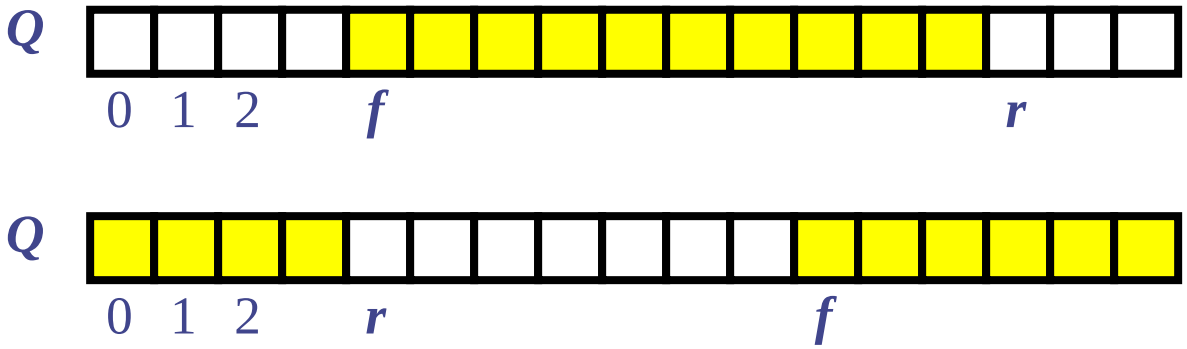
wrapped-around configuration



Queue Operations

- We use the modulo operator (remainder of division)

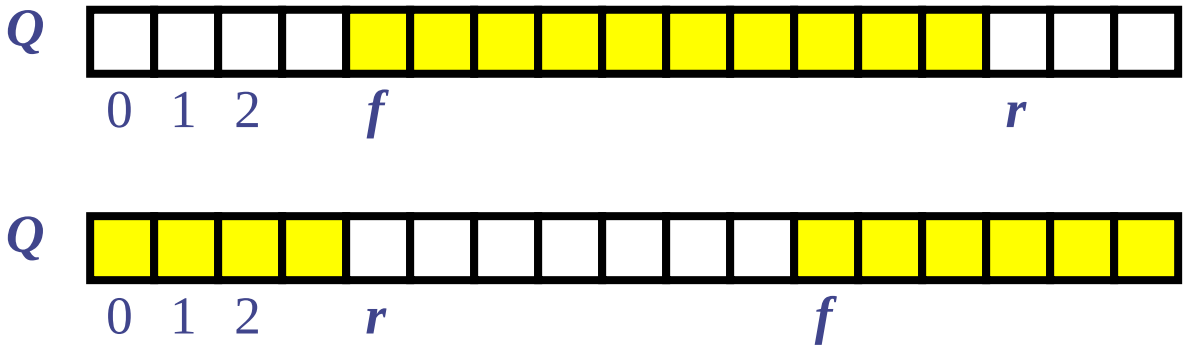
```
Algorithm size()
  return (N + r − f) mod N

Algorithm isEmpty()
  return (f = r)
```



Queue Operations (cont.)

- Operation enqueue throws an exception if the array is full
- This exception is implementation-dependent

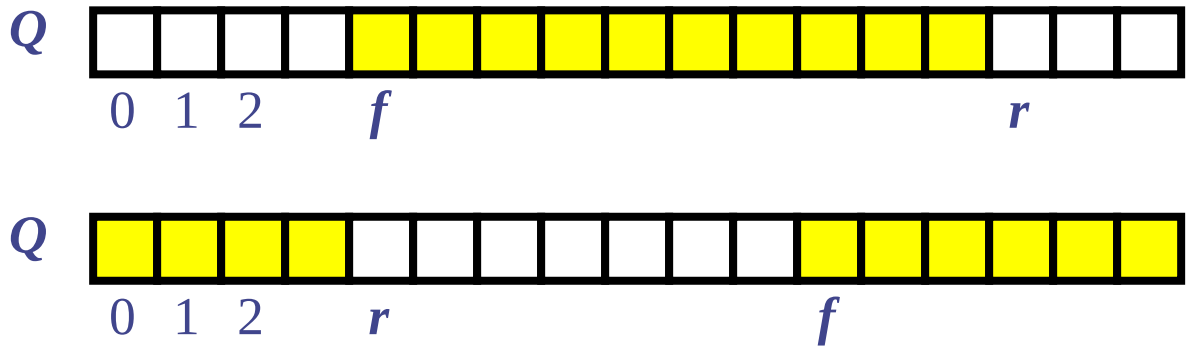
```
Algorithm enqueue(o)
  if size() = N - 1 then
    throw FullQueueException
  else if (f==-1 and r==-1)
    f=0 and r=0
    Q[r] = o
  else
    Q[r] = o
    r = (r + 1) mod N
```



Queue Operations (cont.)

- Operation dequeue throws an exception if the queue is empty
- This exception is specified in the queue ADT

```
Algorithm dequeue()
  if isEmpty() then
    throw EmptyQueueException
   else
    o = Q[f]
    f = (f + 1) mod N
    return o
```



Performance and Limitations

- array-based implementation of queue ADT

- <u>Performance</u>
  - Let $\boldsymbol{n}$ be the number of elements in the queue
  - The space used is $\boldsymbol{O}(\boldsymbol{n})$
  - Each operation runs in time $\boldsymbol{O}(1)$
- <u>Limitations</u>
  - The maximum size of the queue must be defined *apriori* and cannot be changed
  - Trying to enqueue an element into a full queue causes an implementation-specific exception

Growable Array-based Queue

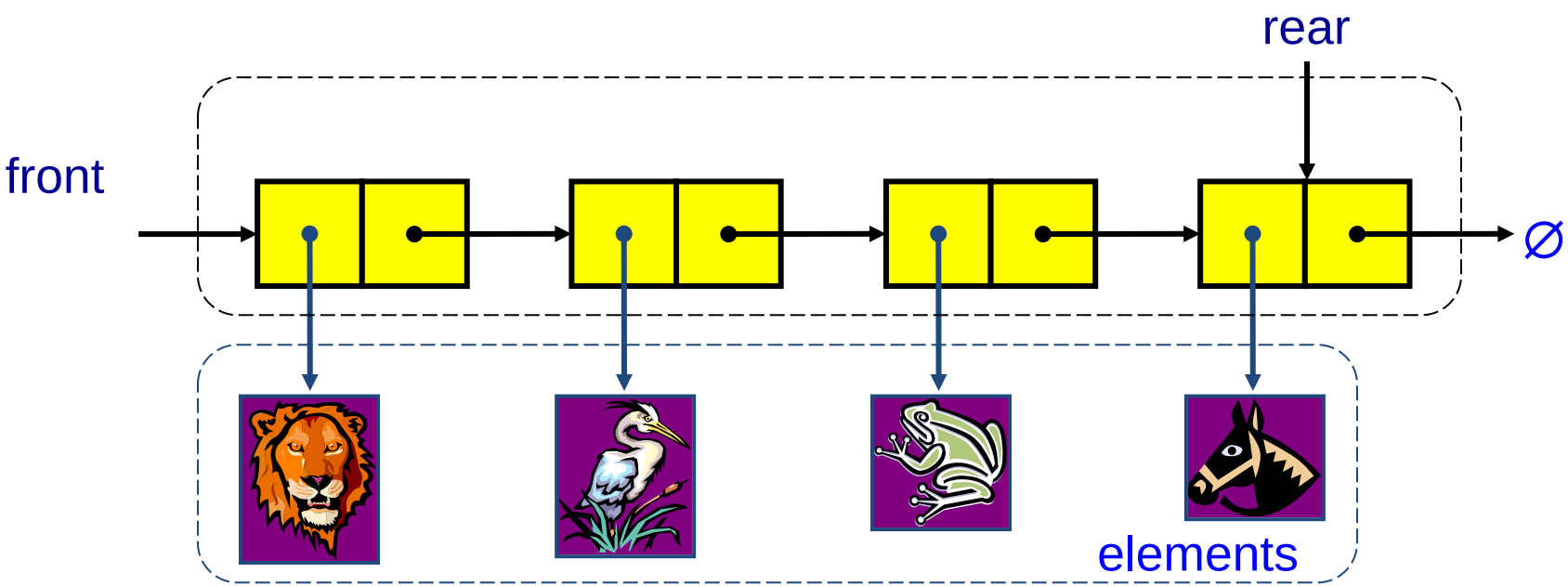
- In an enqueue operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one
- Similar to what we did for an array-based stack
- The enqueue operation has amortized running time
  - $\boldsymbol{O(n)}$ with the incremental strategy
  - $\boldsymbol{O(n)}$ with the doubling strategy

Exercise

- Describe how to implement a queue using a singly-linked list
  - Queue operations: enqueue(x), dequeue(), size(), isEmpty()
  - For each operation, give the running time

Queue with a Singly Linked List

- We can implement a queue with a singly linked list
  - The front element is stored at the head of the list
  - The rear element is stored at the tail of the list
- The space used is $O(n)$ and each operation of the Queue ADT takes $O(1)$ time
- NOTE: we do not have the limitation of the array based implementation on the size of the stack b/c the size of the linked list is not fixed, I.e., the queue is NEVER full.



Informal Queue Interface

- Informal interface for our Queue ADT
- Requires the definition of class EmptyQueueException

```
class Queue {
public:
    int size()
    bool isEmpty();
    Object& front()
            throw(EmptyQueueException)
    void enqueue(Object o)
    Object dequeue()
            throw(EmptyQueueException)
};
```

Linked List Implementation

```c
// Linked list implementation in C

#include <stdio.h>
#include <stdlib.h>

// Creating a node
struct node {
  int value;
  struct node *next;
};

// print the linked list value
void printLinkedlist(struct node *p) {
  while (p != NULL) {
    printf("%d ", p->value);
    p = p->next;
  }
}
```

Linked List Implementation

```
int main() {
  // Initialize nodes
  struct node *head;
  struct node *one = NULL;
  struct node *two = NULL;
  struct node *three = NULL;

  // Allocate memory
  one = malloc(sizeof(struct node));
  two = malloc(sizeof(struct node));
  three = malloc(sizeof(struct node));

  // Assign value values
  one->value = 1;
  two->value = 2;
  three->value = 3;

  // Connect nodes
  one->next = two;
  two->next = three;
  three->next = NULL;

  // printing node-value
  head = one;
  printLinkedlist(head);
}
```

Queue Summary

- Queue Operation Complexity for Different

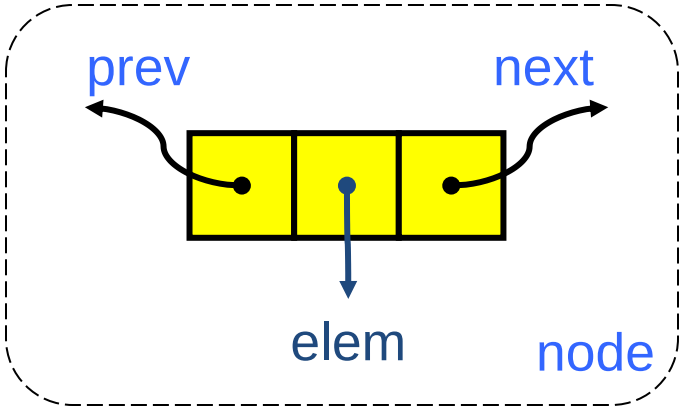
	Array Fixed-Size	Array Expandable (doubling strategy)	List Singly-Linked
dequeue()	O(1)	O(1)	O(1)
enqueue(o)	O(1)	O(n) Worst Case O(1) Best Case O(1) Average Case	O(1)
front()	O(1)	O(1)	O(1)
Size(), isEmpty()	O(1)	O(1)	O(1)

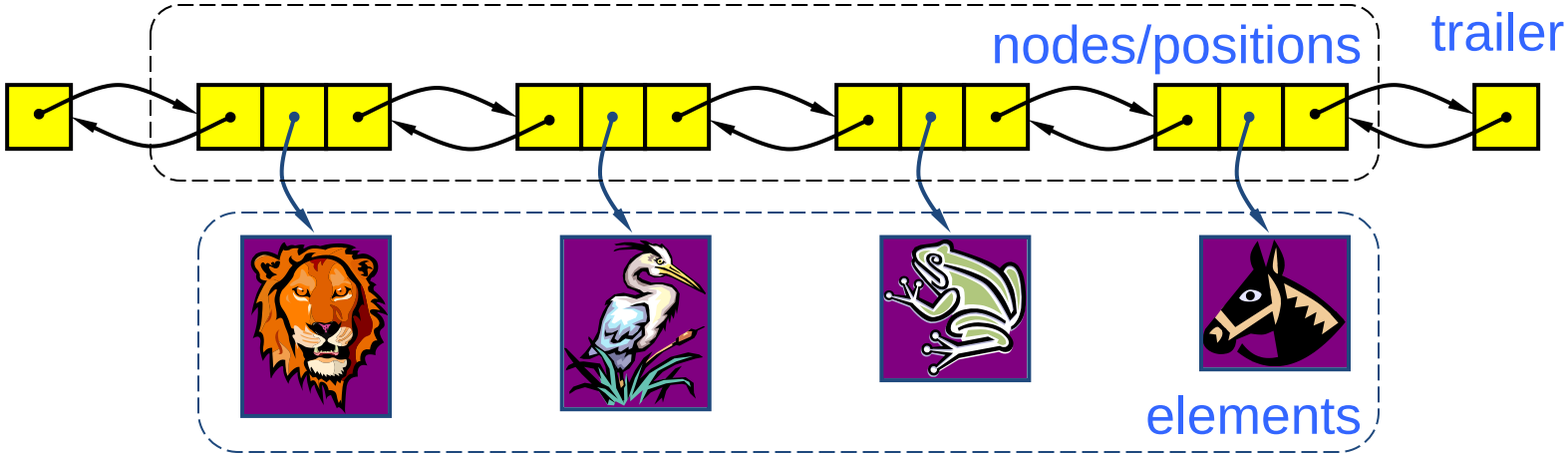
The Double-Ended Queue ADT

- The Double-Ended Queue, or Deque, ADT stores arbitrary objects. (Pronounced 'deck')
- Richer than stack or queue ADTs. Supports insertions and deletions at both the front and the end.
- Main deque operations:
  - insertFirst(object o): inserts element o at the beginning of the deque
  - insertLast(object o): inserts element o at the end of the deque
  - RemoveFirst(): removes and returns the element at the front of the queue
  - RemoveLast(): removes and returns the element at the end of the queue
- Auxiliary queue operations:
  - first(): returns the element at the front without removing it
  - last(): returns the element at the front without removing it
  - size(): returns the number of elements stored
  - isEmpty(): returns a Boolean value indicating whether no elements are stored
- Exceptions
  - Attempting the execution of dequeue or front on an empty queue throws an EmptyDequeException

Doubly Linked List

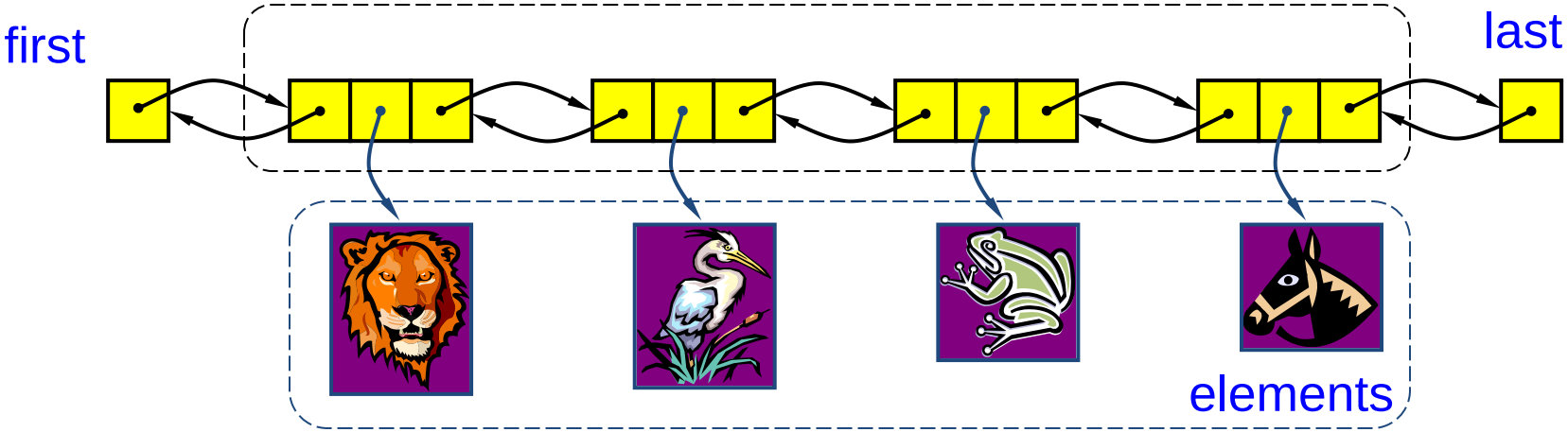
- A doubly linked list provides a natural implementation of the Deque ADT
- Nodes implement Position and store:
  - element
  - link to the previous node
  - link to the next node
- Special trailer and header nodes





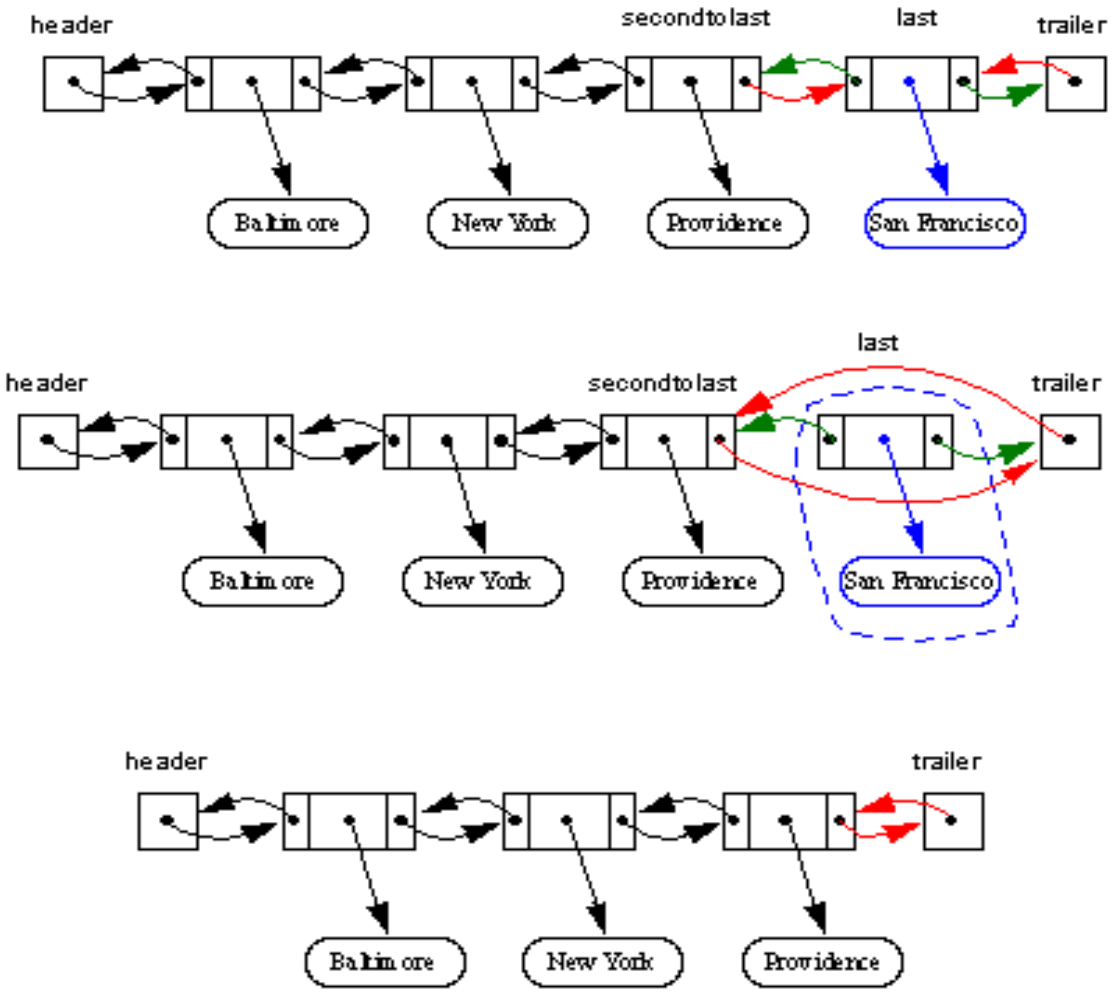
Deque with a Doubly Linked List

- We can implement a deque with a doubly linked list
  - The front element is stored at the first node
  - The rear element is stored at the last node
- The space used is $O(n)$ and each operation of the Deque ADT takes $O(1)$ time



Implementing Deques with Doubly Linked Lists

Here's a visualization of the code for removeLast().



Variants of Deque

- **Input restricted deque**:
  - In this dequeue, insertions can be done only at one of the ends, while deletions can be done from both ends.
- **Output restricted deque**:
  - In this dequeue, deletions can be done only at one of the ends, while insertions can be done on both ends.

Performance and Limitations

- doubly linked list implementation of deque ADT

- Performance
  - Let $\boldsymbol{n}$ be the number of elements in the stack
  - The space used is $\boldsymbol{O}(\boldsymbol{n})$
  - Each operation runs in time $\boldsymbol{O}(1)$
- Limitations
  - NOTE: we do not have the limitation of the array based implementation on the size of the stack b/c the size of the linked list is not fixed, I.e., the deque is NEVER full.

Deque Summary

- Deque Operation Complexity for Different

	Array Fixed-Size	Array Expandable (doubling strategy)	List Singly-Linked	List Doubly-Linked
removeFirst(), removeLast()	O(1)	O(1)	O(1)	O(1)
insertFirst(o), InsertLast(o)	O(1)	O(n) Worst Case O(1) Best Case O(1) Average Case	O(1)	O(1)
first(), last()	O(1)	O(1)	O(1)	O(1)
Size(), isEmpty()	O(1)	O(1)	O(1)	O(1)

Implementing Stacks and Queues with Deques

Stacks with Deques:

Stack Method	Deque Implementation
size()	size()
isEmpty()	isEmpty()
top()	last()
push(e)	insertLast(e)
pop()	removeLast()

Queues with Deques:

Queue Method	Deque Implementation
size()	size()
isEmpty()	isEmpty()
front()	first()
enqueue()	insertLast(e)
dequeue()	removeFirst()

Application of Deque

- For job scheduling applications.
- In O(1) time, we can perform clockwise and anti-clockwise rotational operations.
- Applied as both stack and queue, as it supports both operations.
- Storing a web browser's history.
- Storing a software application's list of undo operations.

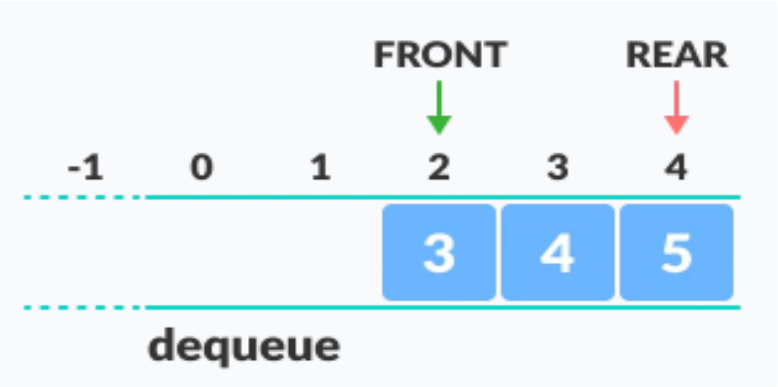
Application of Deque – A-Steal Algorithm

- One example where a deque can be used is the A-Steal job scheduling algorithm.
- This algorithm implements task scheduling for several processors.
- A separate deque with threads to be executed is maintained for each processor.
- To execute the next thread, the processor gets the first element from the deque (using the "remove first element" deque operation).
- If the current thread forks, it is put back to the front of the deque ("insert element at front") and a new thread is executed.
- When one of the processors finishes execution of its own threads (i.e. its deque is empty), it can "steal" a thread from another processor: it gets the last element from the deque of another processor ("remove last element") and executes it.

Source: wikipedia

Circular Queue

- Circular queue avoids the wastage of space in a regular queue implementation using arrays.



- After few enqueue and dequeue, the size of the queue has been reduced.
- The indexes 0 and 1 can only be used after the queue is reset when all the elements have been dequeued.

Circular Queue

- Circular Queue works by the process of circular increment i.e. when we try to increment the pointer and we reach the end of the queue, we start from the beginning of the queue.

- Circular increment is performed by modulo division with queue size
- if REAR + 1 == SIZE (overflow!)

  REAR = (REAR + 1)%SIZE = 0 (start of queue)

Circular Queue Operations

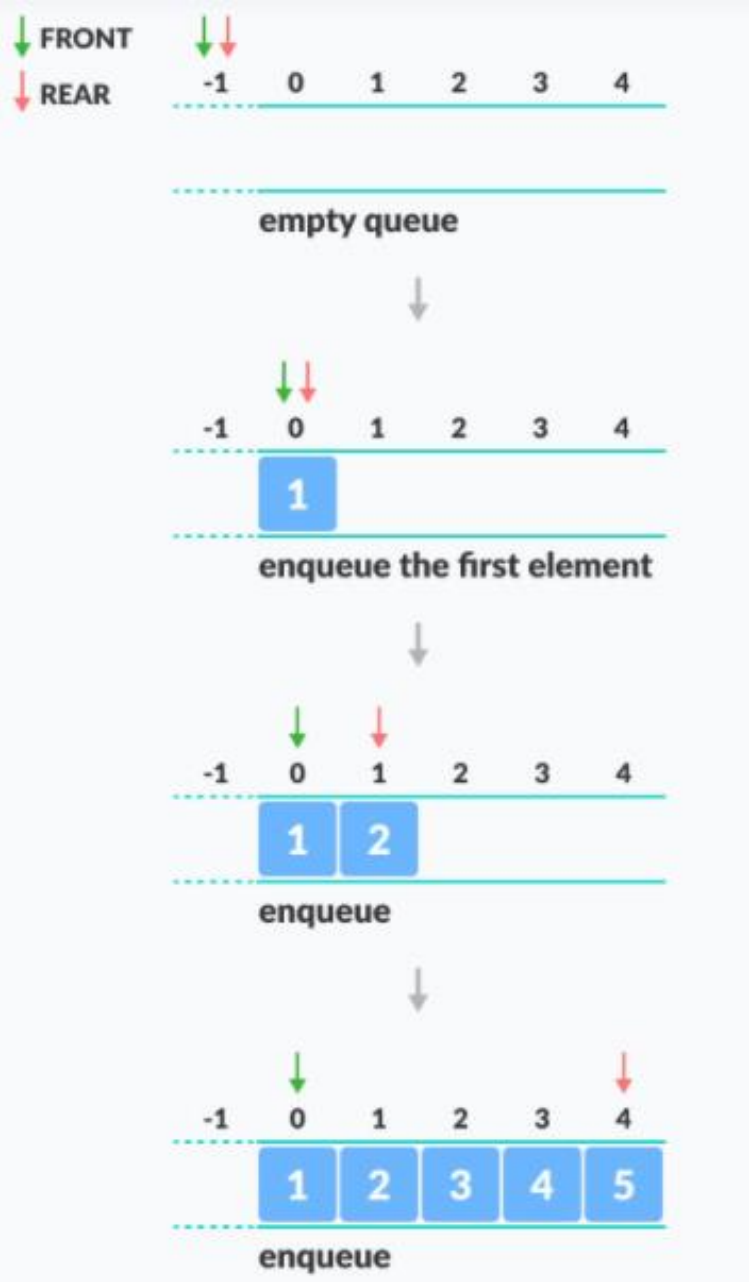
- The circular queue work as follows:
  - two pointers FRONT and REAR
  - FRONT track the first element of the queue
  - REAR track the last elements of the queue
  - initially, set value of FRONT and REAR to -1
- Enqueue Operation
  - check if the queue is full
  - for the first element, set value of FRONT to 0
  - circularly increase the REAR index by 1 (i.e. if the rear reaches the end, next it would be at the start of the queue)
  - add the new element in the position pointed to by REAR

Circular Queue Operations

- Dequeue Operation
  - check if the queue is empty
  - return the value pointed by FRONT
  - circularly increase the FRONT index by 1
  - for the last element, reset the values of FRONT and REAR to -1
- The check for full queue has a new additional case:
  - Case 1: FRONT = 0 && REAR == SIZE - 1
  - Case 2: FRONT = REAR + 1
- The complexity of the enqueue and dequeue operations of a circular queue is O(1) for (array implementations).

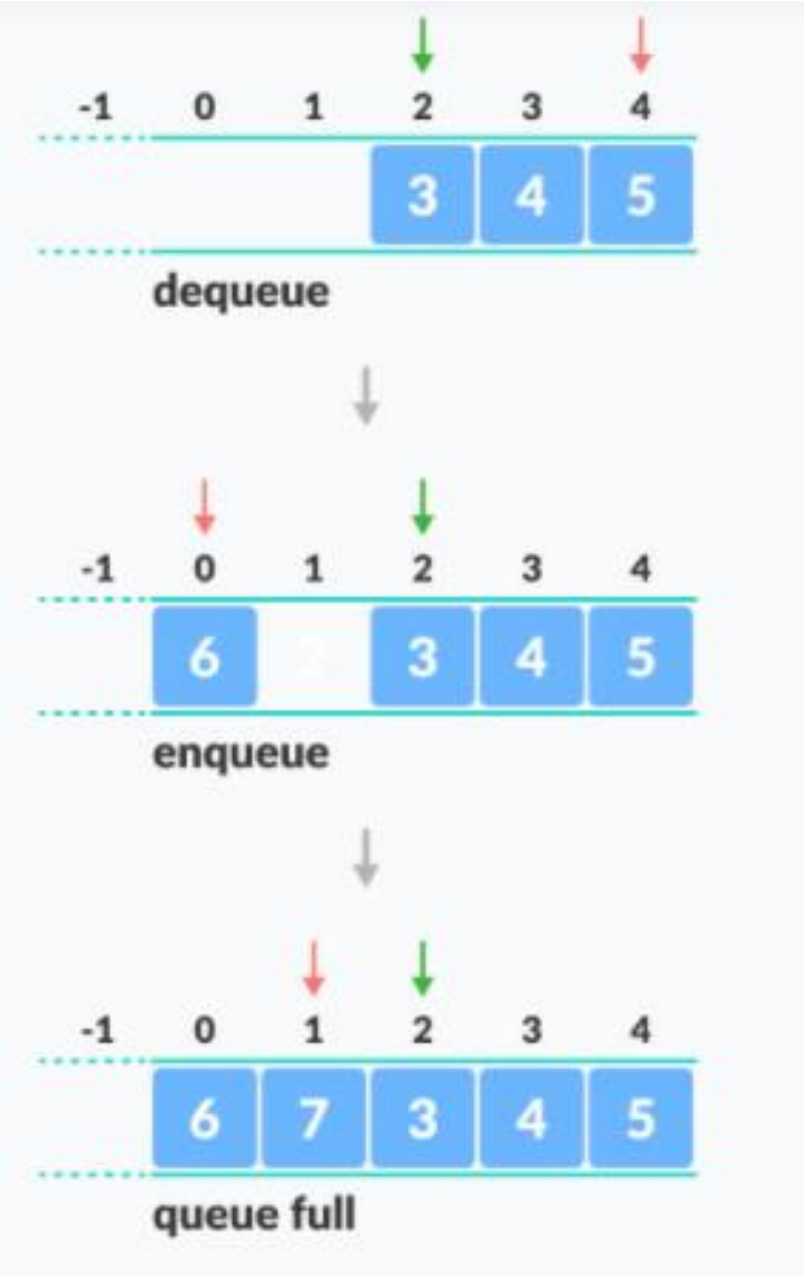
Circular Queue Operations

Case 1:
FRONT = 0 && REAR == SIZE - 1



Circular Queue Operations

Case 2:
FRONT = REAR + 1



Priority Queue

- A priority queue is a special type of queue in which each element is associated with a priority and is served according to its priority.
- If elements with the same priority occur, they are served according to their order in the queue.
- Generally, the element's value is considered for assigning the priority.
- In a queue, the first-in-first-out rule is implemented whereas, in a priority queue, the values are removed on the basis of priority. The element with the highest priority is removed first. **(Max) Priority -> Max-In-First-Out**
- There are two types of priority queue:
  - Ascending order priority queue: lower priority number is given as a higher priority in a priority
  - Descending order priority queue: largest number is given as a higher priority in a priority

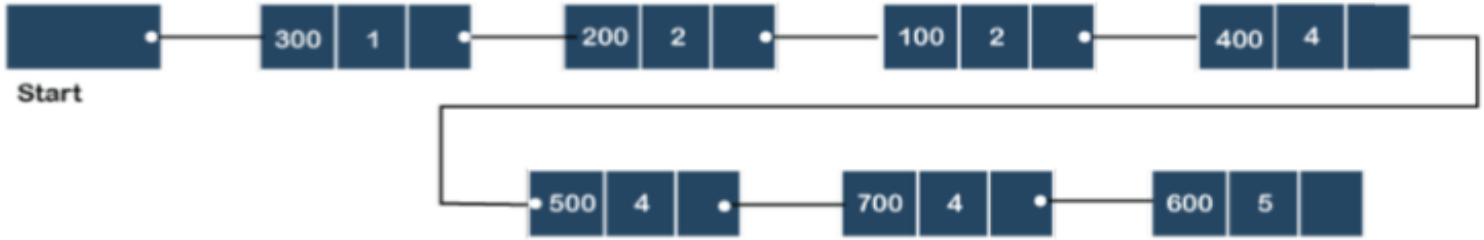
Representation of Priority Queue

	INFO	PNR
0	200	2
1	400	4
2	500	4
3	300	1
4	100	2
5	600	5
6	700	4

- The list given above has INFO list which contains the data elements, PNR contains the priority numbers
- Assumption is lower priority number indicates higher priority

Representation of Priority Queue

1. In the list, lower priority number is 1, whose data value is 300, so it will be inserted in the list
2. Next priority number 2 is having a higher priority, and data values associated with this priority are 200 and 100. So, this data will be inserted based on the FIFO principle; therefore 200 will be added first and then 100.
3. The next higher priority number is 4 and data elements associated with 4 priority numbers are 400, 500, 700. In this case, elements would be inserted based on the FIFO principle; therefore, 400 will be added first, then 500, and then 700.
4. The value associated with priority 5 is 600, so it will be inserted at the end of the queue.



Priority Queue ADT

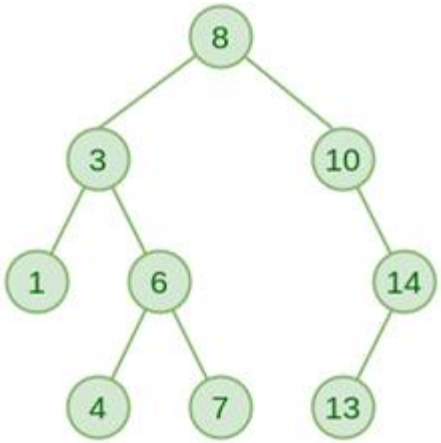
- It is an Abstract Data Type defined by its operations:
  - insert(value, priority) - inserts a value with a certain priority into the data structure
  - deleteMax() - retrieves the value with the highest priority
  - update(value, priority) - changes the priority of the value
  - isEmpty - returns true if the priority queue is empty
- **Example:** Starting from an empty priority queue, insert(A, 1), insert(B, 5), insert(C, 3) followed by deleteMax() gives B and another deleteMax() gives C.

Implementation of Priority Queue

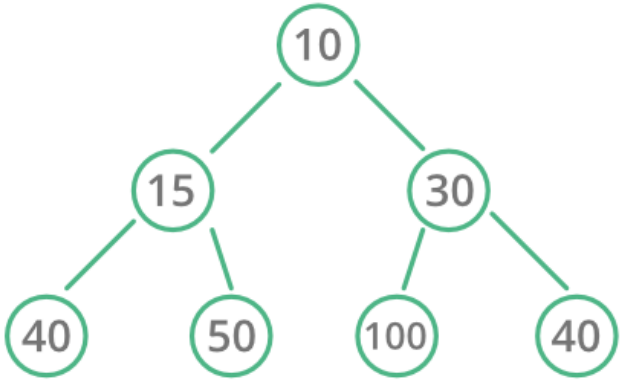
- The priority queue can be implemented in four ways that include arrays, linked list, heap data structure and binary search tree.
- Naïve Implementation: Store in an array
  - Store the elements in the array *unordered*
    - insert(value, priority) - store the element at the end of the array ≈ $O(1)$
    - deleteMax() - find the element with the highest priority, delete it, shift elements to the left by 1 ≈ O(n)
  - Store in an array, *ordered* by priority (the highest at the end):
    - insert(value, priority) - find the position where to store the element, shift to the right the elements starting from that position ≈ $O(n)$
    - deleteMax() - return the last stored element ≈ $O(1)$

Implementation of Priority Queue

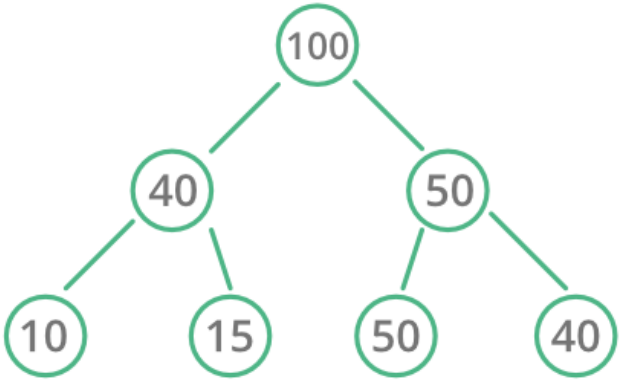
- Better Solution: Store the elements in a binary tree. However, unlike in Binary Search Trees, the highest-priority node is at the top. Called a heap.



Binary Search Tree



Min Heap



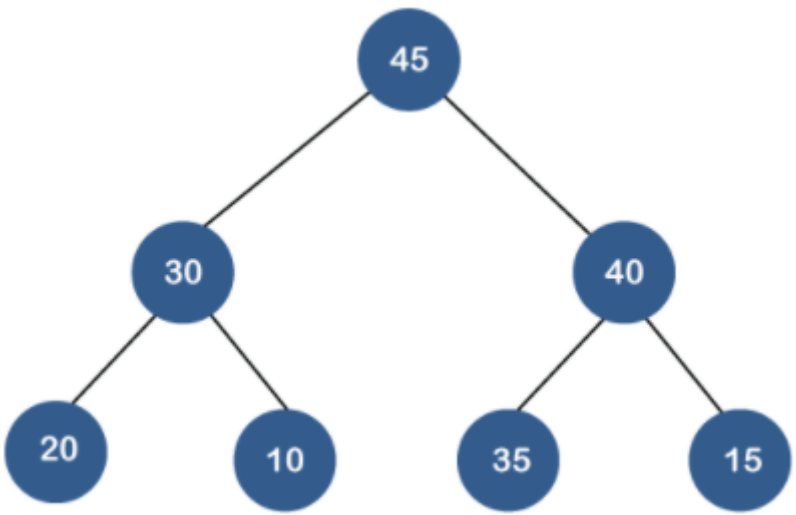
Max Heap

Implementation of Priority Queue

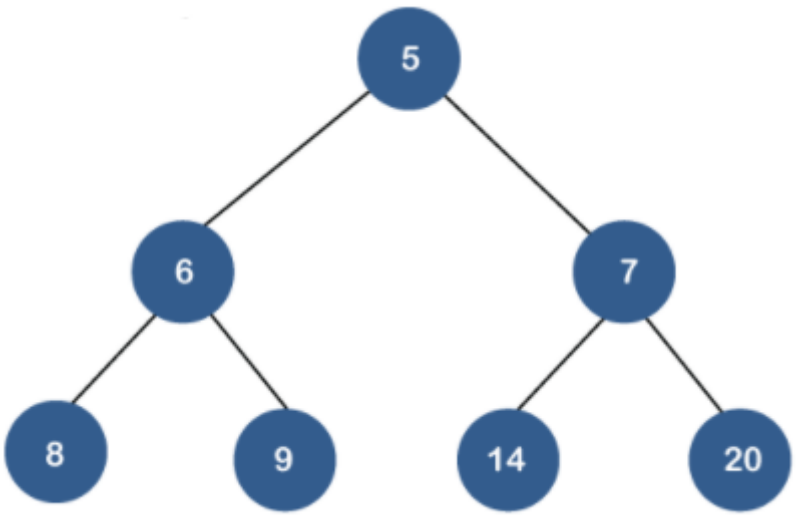
- The heap data structure is the most efficient way of implementing the priority queue
- A heap is a tree-based data structure that forms a complete binary tree, and satisfies the heap property.
- The Heap property
  - The priority of every node is higher than (or equal to) that of its children.
  - All levels are completely filled, except possibly the last one, which is partially filled from the left.
- The height of a heap tree is therefore O(log n)

Implementation of Priority Queue

- Two types of heaps
  - Max heap - Value of parent is greater than that of child
  - Min heap - Value of parent is lesser than that of child



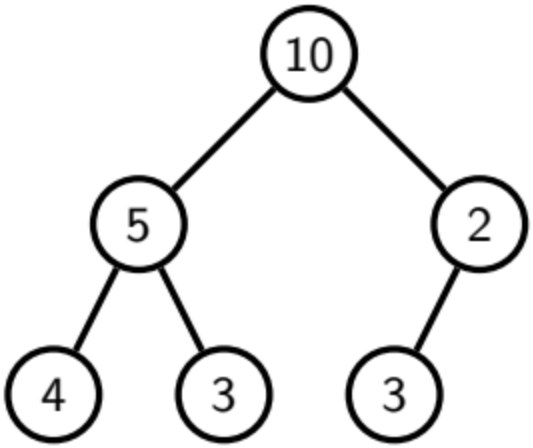
Max Heap



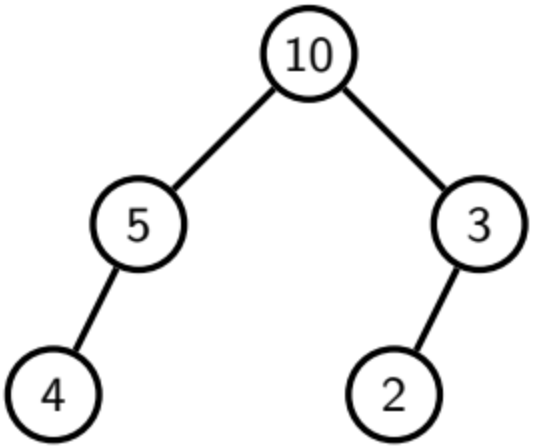
Min Heap

  - We store other things than just the priority but for the sake of understanding of the behaviour of heaps only the priorities are shown.

Examples: Are the following heap trees?



No, because the left child of 2 is 3, that is, a bigger value.

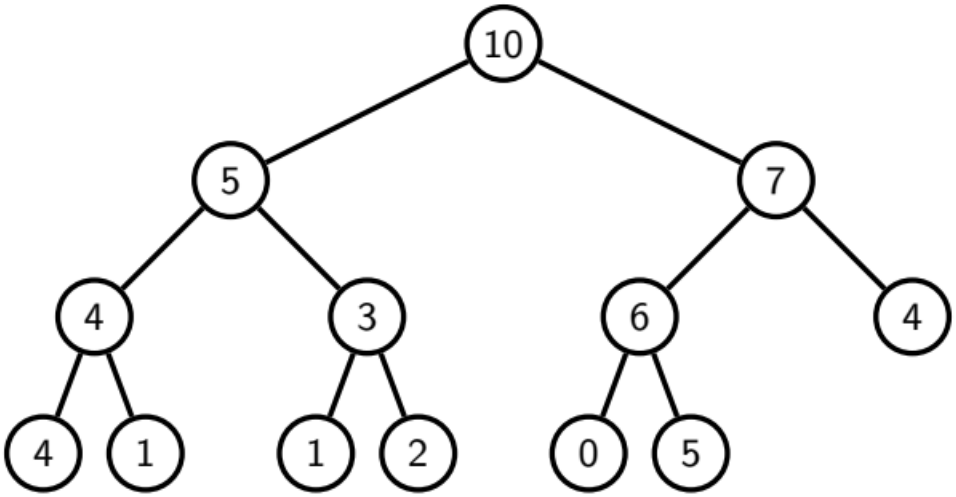


No, because the last layer is not filled from the left.

Insertion in Heap

- **Idea:** Insert the value at the end of the last level and then keep bubbling it up as long as it is larger than its parent.

Insert 8

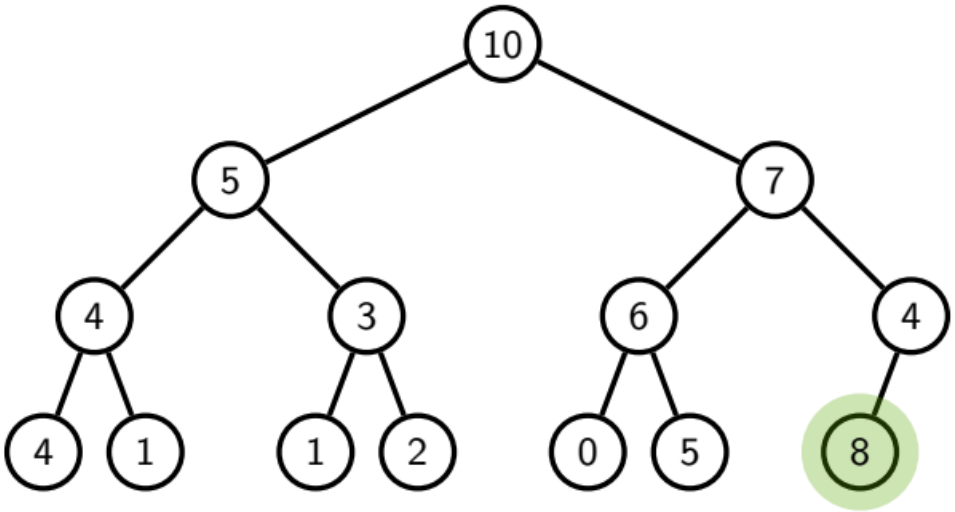


- Initially, we make sure the second condition of heap trees holds, then we fix the first one by bubbling up.

Insertion in Heap

- **Idea:** Insert the value at the end of the last level and then keep bubbling it up as long as it is larger than its parent.

Insert 8

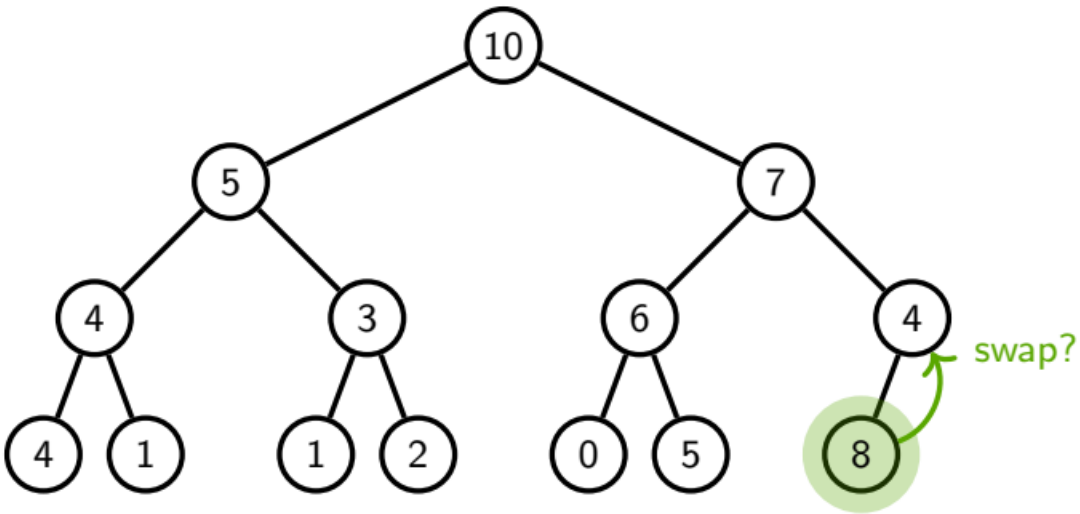


- Initially, we make sure the second condition of heap trees holds, then we fix the first one by bubbling up.

Insertion in Heap

- **Idea:** Insert the value at the end of the last level and then keep bubbling it up as long as it is larger than its parent.

Insert 8

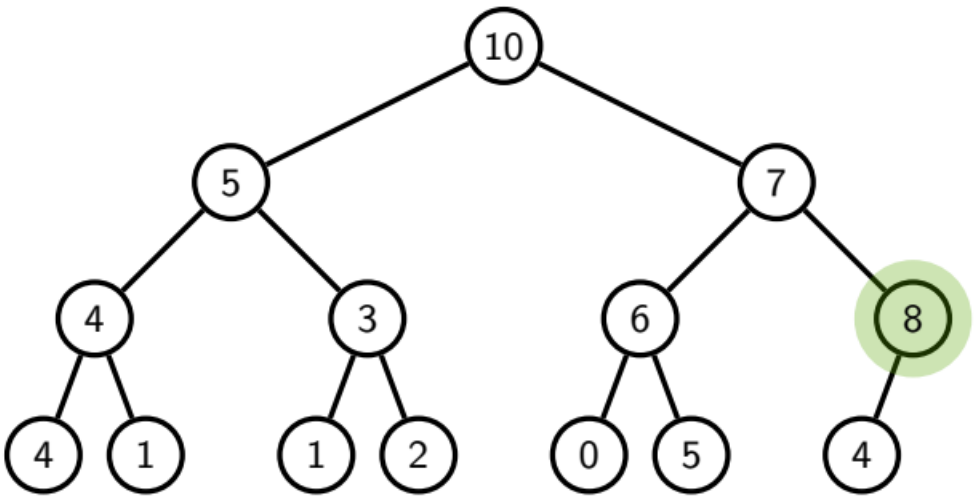


- Initially, we make sure the second condition of heap trees holds, then we fix the first one by bubbling up.

Insertion in Heap

- **Idea:** Insert the value at the end of the last level and then keep bubbling it up as long as it is larger than its parent.

Insert 8

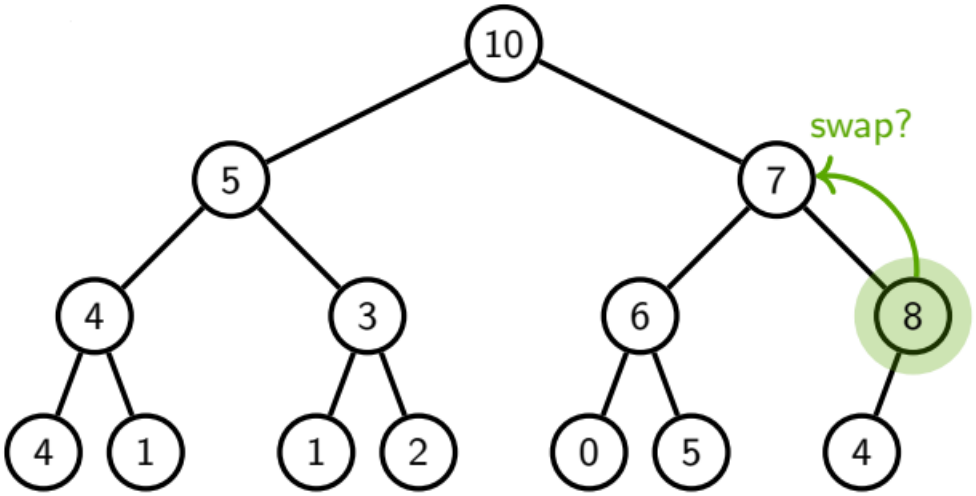


- Initially, we make sure the second condition of heap trees holds, then we fix the first one by bubbling up.

Insertion in Heap

- **Idea:** Insert the value at the end of the last level and then keep bubbling it up as long as it is larger than its parent.

Insert 8

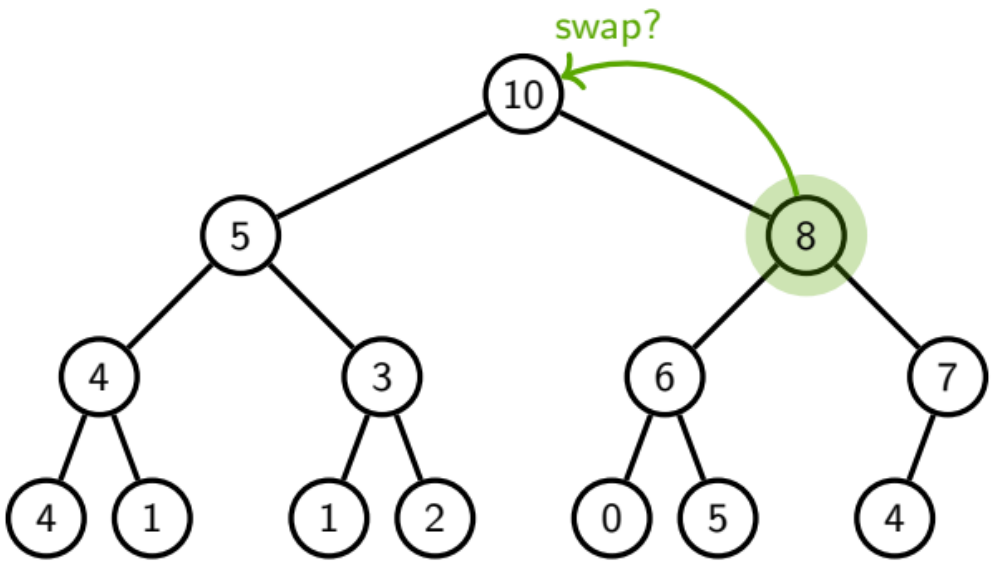


- Initially, we make sure the second condition of heap trees holds, then we fix the first one by bubbling up.

Insertion in Heap

- **Idea:** Insert the value at the end of the last level and then keep bubbling it up as long as it is larger than its parent.

Insert 8

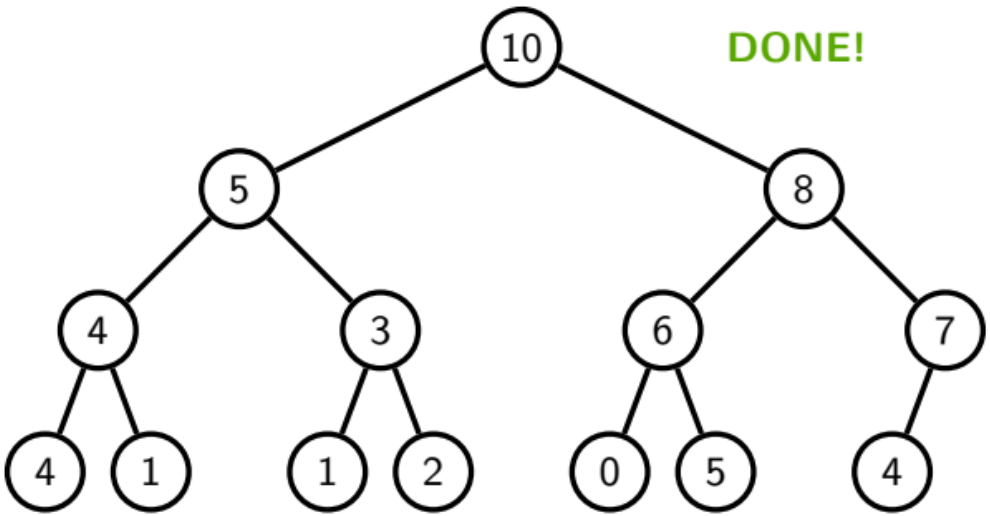


- Initially, we make sure the second condition of heap trees holds, then we fix the first one by bubbling up.

Insertion in Heap

- **Idea:** Insert the value at the end of the last level and then keep bubbling it up as long as it is larger than its parent.

Insert 8

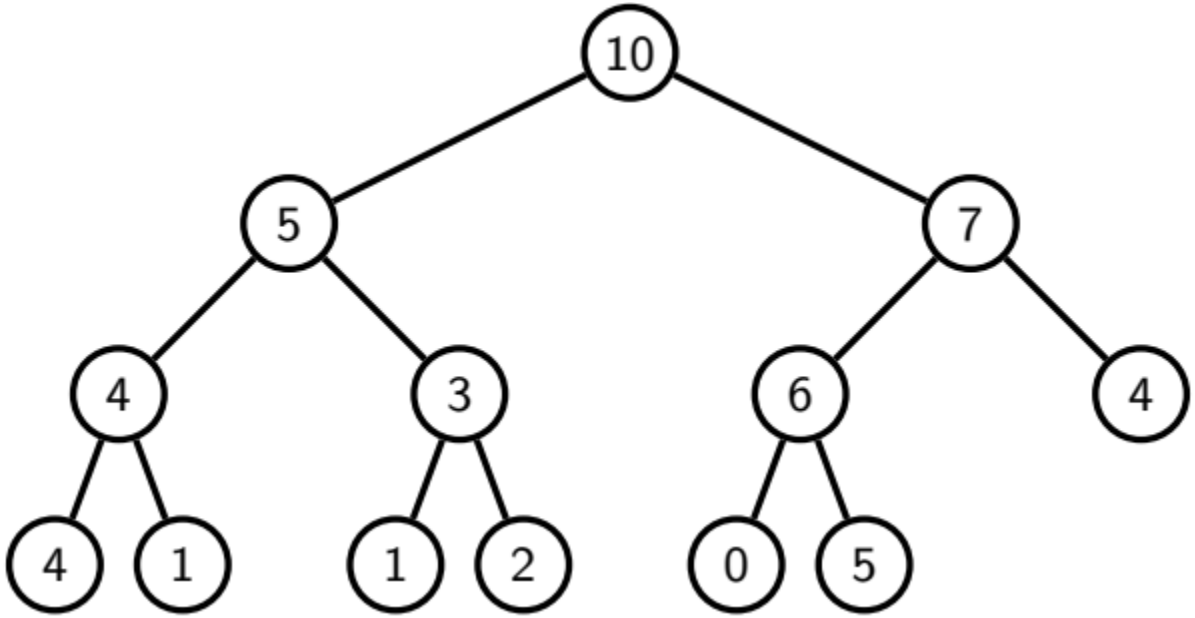


- Initially, we make sure the second condition of heap trees holds, then we fix the first one by bubbling up.

Delete Max in Heap

- **Idea:**
  - Make the last node in the last level the new root.
  - Keep swapping it with the higher priority child as long as any of its children has a higher priority

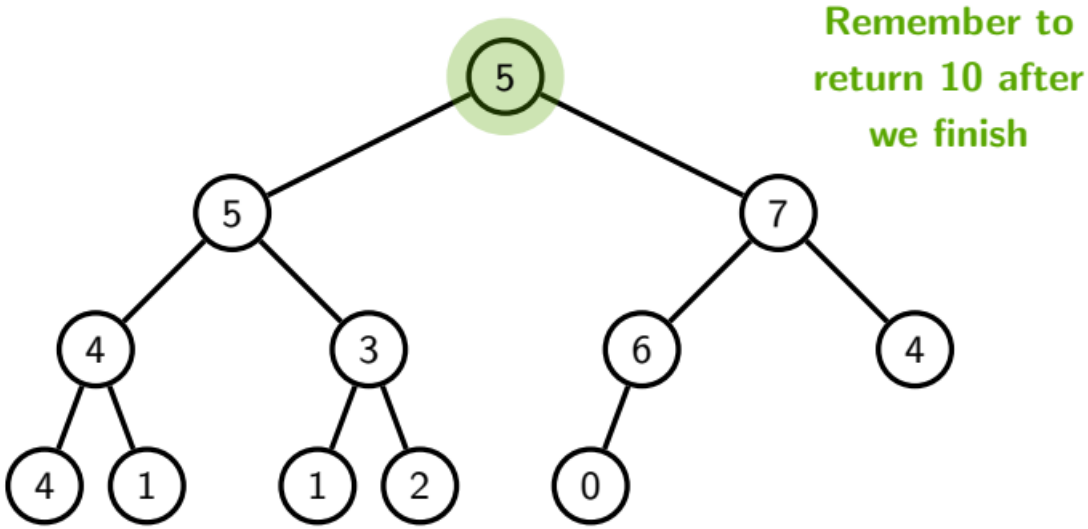
Delete Max



Delete Max in Heap

- **Idea:**
  - Make the last node in the last level the new root.
  - Keep swapping it with the higher priority child as long as any of its children has a higher priority

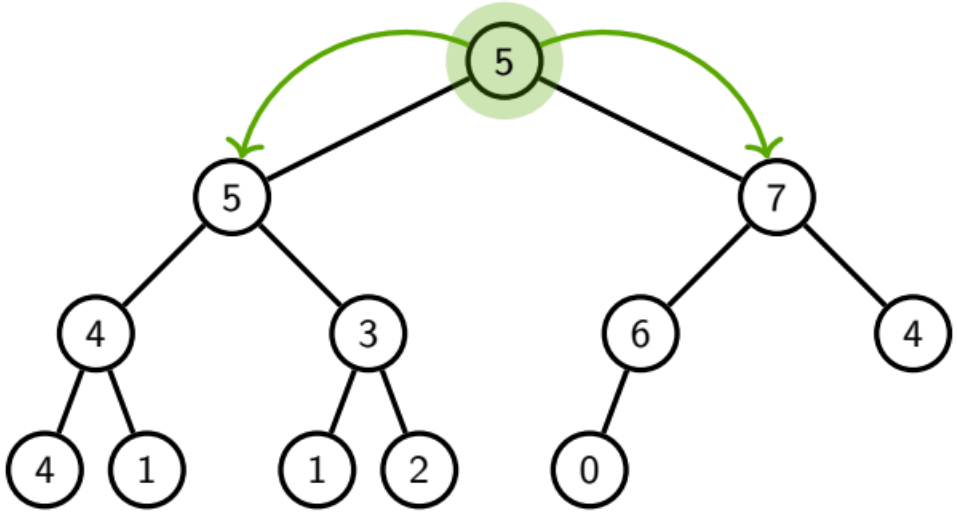
Delete Max



Delete Max in Heap

- **Idea:**
  - Make the last node in the last level the new root.
  - Keep swapping it with the higher priority child as long as any of its children has a higher priority

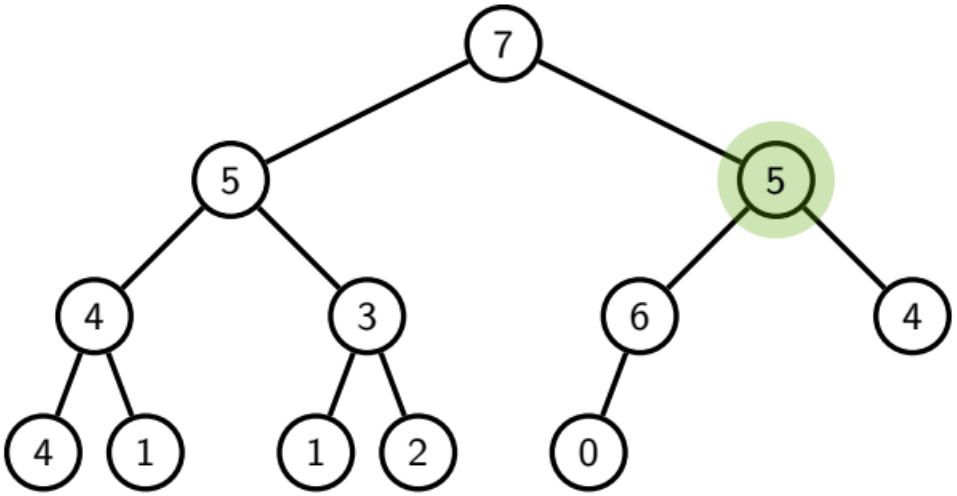
Delete Max



Delete Max in Heap

- **Idea:**
  - Make the last node in the last level the new root.
  - Keep swapping it with the higher priority child as long as any of its children has a higher priority

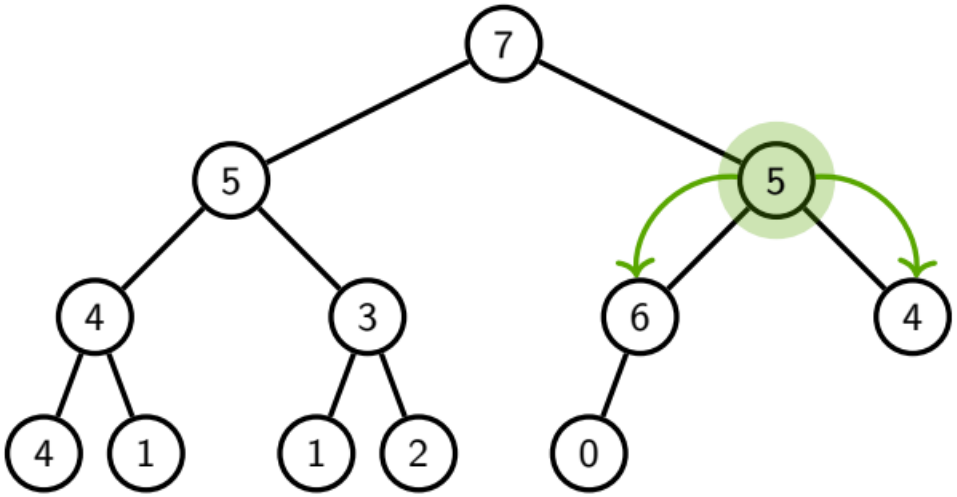
Delete Max



Delete Max in Heap

- **Idea:**
  - Make the last node in the last level the new root.
  - Keep swapping it with the higher priority child as long as any of its children has a higher priority

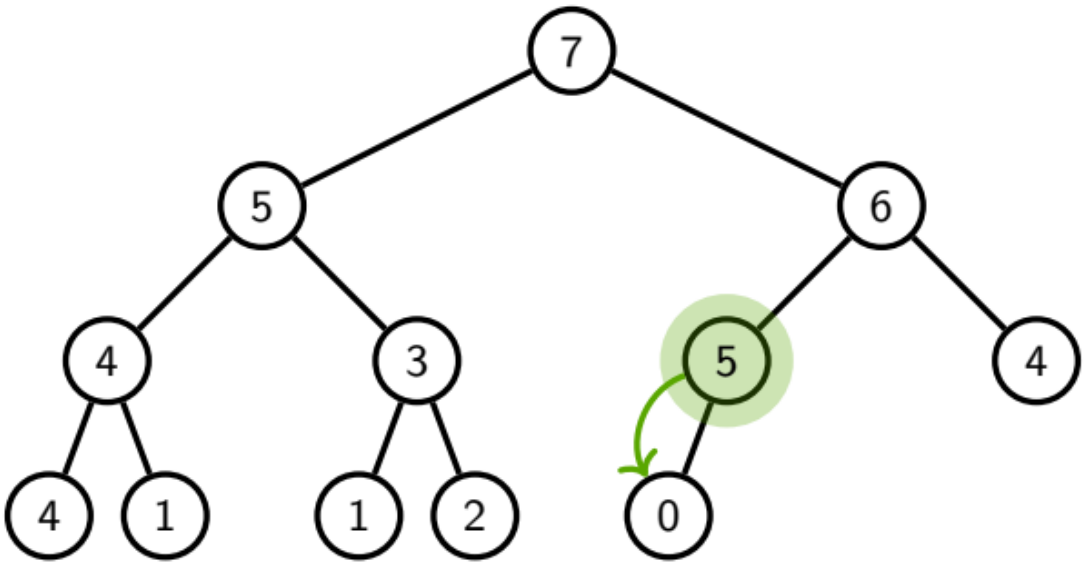
Delete Max



Delete Max in Heap

- **Idea:**
  - Make the last node in the last level the new root.
  - Keep swapping it with the higher priority child as long as any of its children has a higher priority

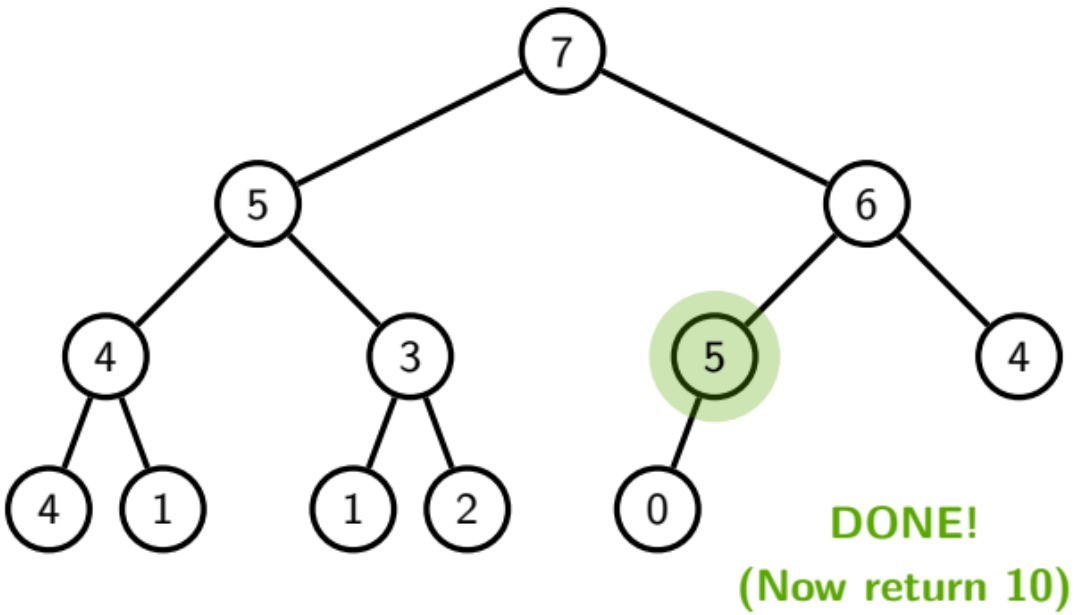
Delete Max



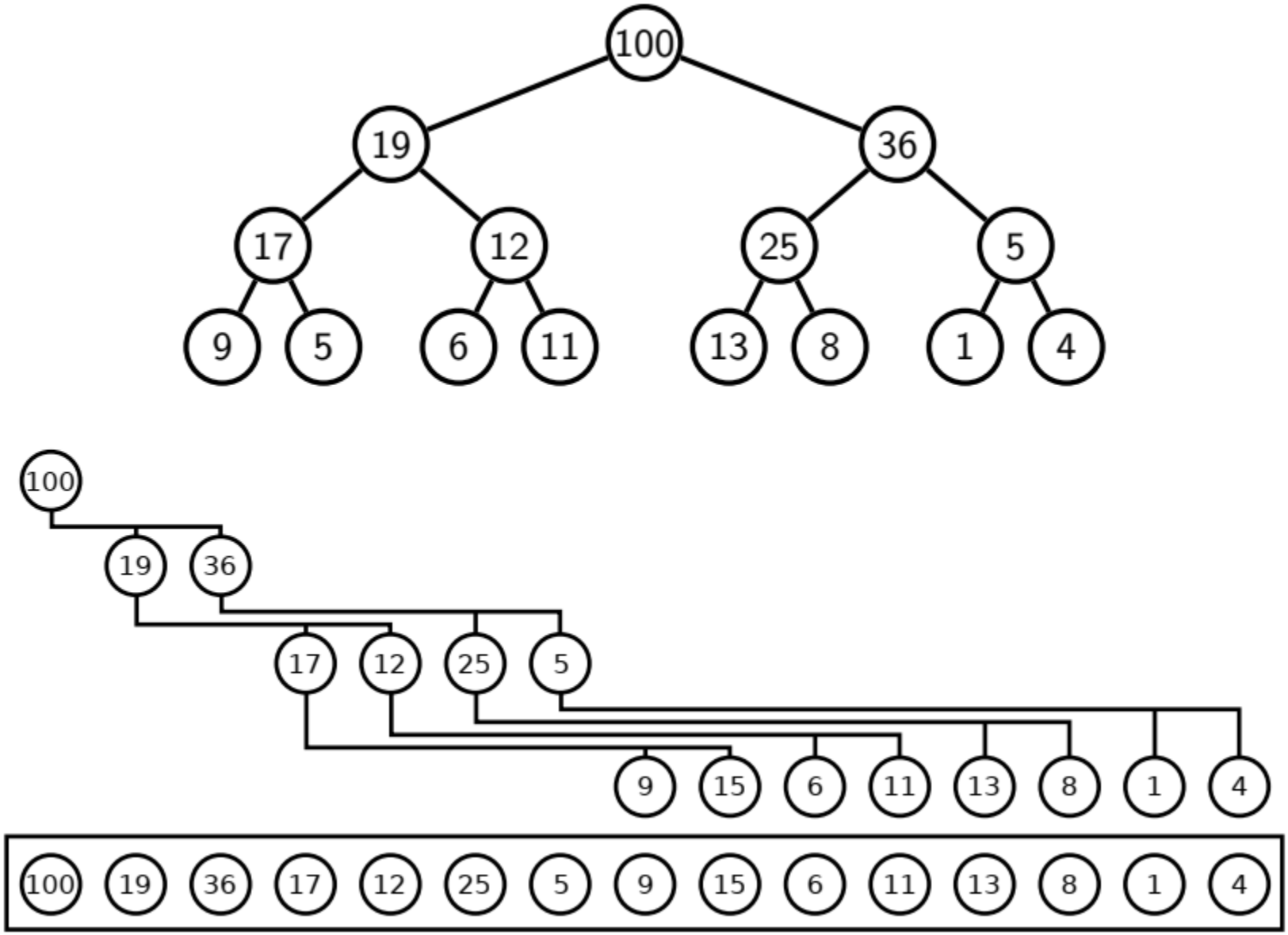
Delete Max in Heap

- **Idea:**
  - Make the last node in the last level the new root.
  - Keep swapping it with the higher priority child as long as any of its children has a higher priority

Delete Max



Storing Heap in Array



Operations on Priority Queue

- Insert(n) - insert element n in the queue
- RemoveMax() - Removing the element of maximum priority
- delete(i) - Removing the element from the priority queue at a given index i
- int get_Max(): Returns the element having the highest priority, and its the root node in max heap.
- int get_Min(): It returns the element having minimum priority, and in max heap, the last node has the smallest value, and lowest priority.

Performance of Operations

Implementation	add	Remove	peek
Linked list	O(1)	O(n)	O(n)
Binary heap	O(logn)	O(logn)	O(1)
Binary search tree	O(logn)	O(logn)	O(1)